Applied Analytical Models
Muhammadjon Merzakulov Najmiddin ugli

Unit 29: Applied Analytical Models

Lecturer: Nilufar Nosirjonova

Group ID: 23-301 DA

Student ID: 230438

Submission Date: 14.06.2026

BTEC LEARNER ASSESSMENT SUBMISSION AND DECLARATION

When submitting evidence for assessment, each learner must sign a declaration confirming that the work is their own.

Learner (Student) ID:	230438
Assessor name:	Nilufar Nosirjonova
BTEC Program Title:	Pearson BTEC Higher Nationals in Digital Technologies
Unit or component number and title:	Unit 29: Applied Analytical Models
Assignment title:	Smart Healthcare Partnership with MediLife Network
Date Assignment submitted:	14.06.2026

Please list the evidence submitted for each task. Indicate the page numbers where the evidence can be found or describe the nature of the evidence (e.g. video, illustration).

Assignment task reference	Evidence submitted

Learner declaration

I certify that the work submitted for this assignment in my own. I have clearly referenced any sources used in the work. I understand that false declaration is a form of malpractice.

Learner Signature:  Date: 14.06.2026

Table of Contents

Introduction **4**

LO1 Examine applied analytical modelling methods. **5**

1.1 Prescriptive and Predictive Analytics *5*

1.2 Supervised vs. Unsupervised Learning *6*

1.3 Comparative Analysis and Business Application *7*

LO2 Prepare a large data set for use in an applied analytical model. **9**

2.1 Data Preparation Process *9*

2.2 Visualising Model Output *9*

2.3 Data Preparation Issues and Transformation Rationale *18*

LO3 Demonstrate the use of an analytical model with a large data set. **24**

3.1 Algorithm Selection and Model Implementation *24*

3.2 Model Evaluation and Comparison *34*

LO4 Investigate improvements to an applied analytical model. **38**

4.1 Investigating Model Improvements *38*

4.2 Performance and Scalability *41*

4.3 Business Presentation of Results *41*

Conclusion **47**

References **48**

Introduction

The rapid growth of social media platforms has created vast streams of behavioural data that organisations are only beginning to exploit analytically. This report investigates the "GenZ Social Media Usage & Mental Health" dataset — an original corpus of **1,000,000 user records** — to answer a single consequential business question: can a user's mental health outcome be reliably predicted from passive digital behaviour signals alone, without any clinical data or psychological assessment?

To address this question, the investigation adopts a dual-track analytical approach. An unsupervised clustering pipeline segments 10,000 users into distinct behavioural archetypes using K-Means, DBSCAN, and Agglomerative Hierarchical Clustering. A parallel supervised regression pipeline trains Ridge Regression, Random Forest, and LightGBM models on 82,393 stratified records to predict the continuous mental_health_score target variable, with a business deployment threshold of $R^2 \geq 0.65$ defined as the minimum acceptable predictive accuracy.

The report is structured across four learning outcomes: examining the theoretical foundations of predictive and prescriptive analytics (LO1); documenting a nine-stage data preparation pipeline including feature engineering, VIF-based multicollinearity filtering, and normalisation (LO2); implementing and evaluating all six algorithms with quantified metrics (LO3); and investigating model improvements, scalability strategies, and business recommendations (LO4). All code, Jupyter Notebooks, and model outputs are publicly available at: **https://github.com/muhammadjon-merzaqulov/applied_models**

LO1 Examine applied analytical modelling methods.

1.1 Prescriptive and Predictive Analytics

Predictive Analytics

Definition and Business Purpose: Predictive analytics is the practice of using historical data, statistical algorithms, and machine learning techniques to forecast future outcomes. Its primary purpose in a business context is to reduce uncertainty in decision-making by identifying patterns in past data and projecting them forward. Rather than explaining what happened, predictive analytics answers the question: *what is likely to happen next?*

Real-World Example — Healthcare: In healthcare, predictive analytics is widely applied for early disease detection. A hospital might analyse patient records — including age, lab results, symptoms, and exposure history — to predict the likelihood of a patient developing a severe illness. In the context of this investigation, the Hantavirus dataset is used to predict whether a patient tests **Hantavirus positive** (Hantavirus_Positive = 1) based on clinical features such as WBC count, CRP levels, creatinine, and symptom count. This allows clinicians to prioritise high-risk patients before confirmatory test results arrive.

Suitable Algorithm: Random Forest Classifier — an ensemble supervised learning method well-suited to tabular medical data with mixed feature types.

Prescriptive Analytics

Definition and Business Purpose: Prescriptive analytics goes beyond prediction by recommending specific actions to achieve a desired outcome. It uses optimisation algorithms, simulation, and decision logic to answer: *what should we do?* In a business context, it supports resource allocation, policy design, and operational strategy by prescribing the best course of action given predicted conditions.

Real-World Example — Logistics: Amazon's delivery routing system uses prescriptive analytics to assign packages to drivers, sequence delivery routes, and schedule warehouse dispatches in real time. Given predicted demand and traffic data, the system prescribes the most cost-efficient route for each driver — reducing fuel costs and improving delivery speed simultaneously.

Suitable Algorithm: Linear Programming / Genetic Algorithms — optimisation techniques that find the best solution from a defined set of constraints and objectives.

Three Analytical Methods: How They Function

Method 1 — Random Forest (Predictive, Supervised): Random Forest operates by constructing a large number of decision trees during training, each built on a random subset of the data and a random subset of features. At prediction time, each tree independently classifies the input, and the majority vote across all trees is taken as the final output. This bagging approach reduces variance and prevents overfitting compared to a single decision tree.

- **Inputs:** Numerical and encoded categorical features (e.g., WBC count, age, region, symptom count)
- **Logic:** Bootstrap aggregation (bagging) — multiple trees trained on random samples, combined by majority voting
- **Outputs:** Class label (e.g., *Hantavirus_Positive = 0 or 1*) and feature importance scores

Method 2 — K-Means Clustering (Unsupervised): K-Means partitions a dataset into *k* clusters by iteratively assigning each data point to the nearest cluster centroid, then recalculating centroids based on the mean of all assigned points. The process repeats until centroids stabilise (convergence). The algorithm minimises the Within-Cluster Sum of Squares (WCSS), meaning points within each cluster are as similar as possible.

- **Inputs:** Numerical features (e.g., *daily_usage_hours*, *avg_session_minutes*, *mental_health_score*) — no labels required
- **Logic:** Euclidean distance from each point to cluster centroids, iterative assignment and update
- **Outputs:** Cluster assignments for each data point, centroid coordinates, WCSS value

Method 3 — Logistic Regression (Predictive, Supervised): Despite its name, Logistic Regression is a classification algorithm. It models the probability that a given input belongs to a particular class by applying a sigmoid (logistic) function to a linear combination of input features. The output is a probability between 0 and 1, and a threshold (typically 0.5) is applied to produce a binary class label.

- **Inputs:** Numerical features and encoded categorical variables
- **Logic:** Weighted linear combination of features passed through sigmoid function: P(y=1) = 1 / (1 + e^−z)
- **Outputs:** Probability score and binary class prediction (e.g., positive/negative for Hantavirus)

1.2 Supervised vs. Unsupervised Learning

Supervised Learning

Supervised learning is a category of machine learning in which a model is trained on a labelled dataset — meaning each training example is paired with a known output (target variable). The model learns the mapping between input features and output labels, then applies this learned relationship to predict outcomes on new, unseen data. The term "supervised" reflects the fact that the correct answers guide the learning process.

Examples of supervised learning algorithms:

- **Linear Regression** — predicts a continuous numerical output (e.g., sales forecast)
- **Logistic Regression** — predicts binary or multi-class categorical outcomes
- **Decision Tree** — partitions data by feature thresholds in a tree-like structure
- **Random Forest** — ensemble of decision trees using bagging
- **k-Nearest Neighbours (k-NN)** — classifies based on the majority label among k nearest data points
- **Support Vector Machine (SVM)** — finds the optimal hyperplane separating classes
- **Neural Networks** — multi-layered networks capable of learning complex non-linear patterns

In this investigation, the **Hantavirus dataset** is used for supervised learning. The target variable is *Hantavirus_Positive* (binary: 0 or 1), and the model learns from clinical features such as WBC count, platelet count, CRP levels, ALT, AST, creatinine, and symptom count to classify whether a patient tests positive.

Unsupervised Learning

Unsupervised learning involves training a model on data that has **no predefined labels or target variable**. The algorithm identifies hidden structure, patterns, or groupings within the data entirely on its own. Rather than predicting an outcome, the goal is to discover the natural organisation of the data.

Examples of unsupervised learning algorithms:

- **K-Means Clustering** — groups data points into k clusters based on distance to centroids
- **Hierarchical Clustering** — builds a tree (dendrogram) of nested clusters
- **DBSCAN** — density-based clustering that identifies arbitrary cluster shapes and outliers
- **PCA (Principal Component Analysis)** — reduces dimensionality while preserving variance
- **Apriori / Association Rules** — discovers item co-occurrence patterns (e.g., market basket analysis)

In this investigation, the **Social Media Usage dataset (Gen Z)** is used for unsupervised learning. K-Means clustering is applied to features such as *daily_usage_hours*, *avg_session_minutes*, *mental_health_score*, *num_platforms_used*, and *screen_time_before_sleep*. The objective is to discover natural user behaviour segments without imposing predefined group labels.

Business Scenarios Best Suited to Each Type

Scenario	Learning Type	Reason
Predicting patient disease outcome	Supervised	Known historical diagnoses serve as labels
Detecting credit card fraud	Supervised	Labelled fraud/non-fraud transaction history available
Classifying email as spam or not spam	Supervised	Pre-labelled training examples exist
Customer segmentation for a new platform	Unsupervised	No predefined customer categories; patterns must be discovered
Anomaly detection in network traffic	Unsupervised	Normal behaviour must be learned without fraud labels
Grouping social media users by behaviour	Unsupervised	No ground-truth segments exist; discovery is the goal
Product recommendation from purchase history	Unsupervised	Association rules applied without explicit labels

1.3 Comparative Analysis and Business Application

Comparison of Predictive and Prescriptive Models

Dimension	Predictive Analytics	Prescriptive Analytics

Core Question	What will happen?	What should we do about it?
Output Type	Forecast, probability, classification	Recommended action, optimised decision
Data Requirement	Historical labelled or unlabelled data	Predictive output + constraints + objectives
Algorithms Used	Regression, classification, clustering	Optimisation, simulation, reinforcement learning
Complexity	Moderate	High
Business Value	Reduces uncertainty	Drives direct action
Advantage	Actionable foresight; reduces risk	Automates optimal decision-making
Disadvantage	Does not recommend action; may be wrong	Requires accurate predictions as input; computationally expensive
Example	Predicting which patients are Hantavirus positive	Prescribing which patients should receive priority treatment first

Both approaches are complementary: predictive models generate forecasts, and prescriptive models consume those forecasts to determine the best response. In practice, the most sophisticated organisations implement both in sequence.

Case Study: Google — DeepMind AI for Data Centre Cooling

Background: Google's data centres consume enormous amounts of electricity, a significant portion of which is used for cooling server infrastructure. In 2016, Google deployed DeepMind's reinforcement learning system to manage the cooling systems across its data centres, replacing traditional rule-based controls.

The Analytical Model Applied: DeepMind trained a neural network on thousands of sensor readings collected from the data centres — including temperature, power consumption, pump speeds, and cooling settings. The model learned to predict the future energy efficiency (PUE — Power Usage Effectiveness) of the facility given current conditions (predictive layer). A reinforcement learning agent then used these predictions to recommend specific control actions — fan speeds, cooling water temperatures, and equipment setpoints — in real time (prescriptive layer).

Impact on Decision-Making: Prior to the AI system, cooling decisions were made by human engineers following fixed schedules and manual heuristics. DeepMind's system replaced this with data-driven, continuous optimisation. According to Google's published findings, the AI achieved a 40% reduction in cooling energy usage, representing a 15% overall improvement in data centre energy efficiency. Decision-making moved from periodic human intervention to autonomous, millisecond-level adjustments.

Operational Efficiency: The system operates 24 hours a day without fatigue, responding to real-time environmental changes faster than any human operator could. It evaluates hundreds of variable combinations simultaneously — something impossible manually. This enabled Google to reduce its total data centre energy consumption substantially, lowering both operating costs and carbon emissions.

Competitive Advantage: Google's ability to reduce energy costs through AI directly improved its profit margins on cloud services, allowing it to offer competitive pricing on Google Cloud Platform. Furthermore, the demonstrated capability of the DeepMind system established Google as a leader in industrial AI application — a significant reputational and commercial advantage. The technology has since been licensed and applied by other organisations, creating an additional revenue stream.

Critical Evaluation: The DeepMind case demonstrates how combining predictive and prescriptive analytics creates value that neither approach could achieve alone. The predictive model provided accurate short-term forecasting of energy efficiency, while the prescriptive reinforcement learning agent acted on those forecasts to optimise in real time. However, the system also illustrates key limitations: it required years of historical sensor data, significant engineering investment, and careful safety constraints to prevent the AI from making dangerous hardware decisions. There are also questions of explainability — it is difficult for engineers to understand *why* the model recommends a specific cooling setting, which raises concerns about trust and accountability in safety-critical environments. Despite these limitations, the measurable efficiency gains and the scale at which the system operates make this one of the most compelling real-world applications of applied analytical modelling in business.

LO2 Prepare a large data set for use in an applied analytical model.

2.1 Data Preparation Process

Preparing a large dataset for analytical modelling is the most critical and time-consuming stage of the entire pipeline. The principle of "garbage in, garbage out" directly applies: the reliability of every model output is fundamentally determined by the quality of the data fed into it. This section documents the full data preparation pipeline applied to the "GenZ Social Media Usage & Mental Health" dataset, which originally contained **1,000,000 records** and **12 columns**: age, gender, country, daily_usage_hours, primary_platform, num_platforms_used, purpose, avg_session_minutes, night_usage, mental_health_score, addiction_level, and screen_time_before_sleep. All Python code implementing this pipeline is available in the project GitHub repository.

Stage 1 — Sampling

Processing all 1,000,000 rows simultaneously in memory-intensive algorithms such as hierarchical clustering or gradient boosting would cause severe RAM bottlenecks on standard computing environments. Two sampling strategies were therefore applied, one for each modelling task.

For supervised regression, a **stratified sampling** strategy was used to ensure that the target variable mental_health_score was evenly represented across all score ranges. The full score range (1.00–10.00) was divided into 5 equal-width bins, and 20,000 rows were drawn from each bin, producing a final supervised sample of **82,393 rows** (8.2% of the original dataset). The bin distribution after sampling was confirmed as balanced:

- Bin 0 (lowest scores): 2,393 rows
- Bins 1–4: 20,000 rows each

```
Stratified Sampling complete:
  Original dataset :  1,000,000 rows
  Sampled dataset  :     82,393 rows  (8.2% of original)
  Bins used        : 5
  Rows per bin     : ~20000
  Score range      : [1.00 – 10.00]

Rows per bin after sampling (should be roughly equal):
  Bin 0: 2,393
  Bin 1: 20,000
  Bin 2: 20,000
  Bin 3: 20,000
  Bin 4: 20,000
```

	age	gender	country	daily_usage_hours	primary_platform	num_platforms_used	purpose	avg_session_minutes	night_usage	mental_health_score	addiction_level	screen_time_before_sleep
0	22	Male	India	6.904213	Instagram	2	Entertainment	32.789215	1	4.626507	High	65.856646
1	16	Female	Germany	6.185238	Instagram	5	Entertainment	25.346714	1	4.189395	High	21.762819
2	21	Female	USA	4.488727	Instagram	4	Socializing	17.535582	1	6.243987	Medium	51.799210
3	15	Female	India	0.981497	TikTok	1	News	24.135902	1	9.596209	Low	12.241502
4	17	Male	UK	7.459372	Snapchat	1	Entertainment	31.394755	1	2.853412	High	57.732774

Figure 1: Stratified sampling output

For unsupervised clustering, a **random sample of 10,000 rows** (1% of the dataset) was drawn using DataFrame.sample(n=10_000, random_state=42). Random sampling (rather than stratified) is appropriate here because unsupervised learning has no target label, so there is no variable to balance across — the aim is simply to preserve the natural distribution of all behavioural features equally.

Stage 2 — Exploratory Data Analysis (EDA)

Before any transformation, descriptive statistics were computed on the supervised sample (82,393 rows) to understand the structure and distribution of the dataset.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 82393 entries, 0 to 82392
Data columns (total 12 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   age                      82393 non-null  int64
 1   gender                   82393 non-null  object
 2   country                  82393 non-null  object
 3   daily_usage_hours        82393 non-null  float64
 4   primary_platform         82393 non-null  object
 5   num_platforms_used       82393 non-null  int64
 6   purpose                  82393 non-null  object
 7   avg_session_minutes      82393 non-null  float64
 8   night_usage              82393 non-null  int64
 9   mental_health_score      82393 non-null  float64
 10  addiction_level          82393 non-null  object
 11  screen_time_before_sleep  82393 non-null  float64
dtypes: float64(4), int64(3), object(5)
memory usage: 7.5+ MB


                age  daily_usage_hours  num_platforms_used  avg_session_minutes  \
count  82393.0000         82393.0000          82393.0000           82393.0000
mean      19.9864             4.0925              2.9988              25.0408
std        4.3132             1.7777              1.4140               9.7622
min       13.0000             0.5000              1.0000               5.0000
25%       16.0000             2.7718              2.0000              18.2089
50%       20.0000             4.1015              3.0000              24.9673
75%       24.0000             5.3927              4.0000              31.7189
max       27.0000            10.0000              5.0000              67.6071

       night_usage  mental_health_score  screen_time_before_sleep
count   82393.0000           82393.0000                82393.0000
mean        0.5953               6.3920                   40.1927
std         0.4908               2.0272                   19.6696
min         0.0000               1.0000                    0.0000
25%         0.0000               4.5348                   26.4326
50%         1.0000               6.3304                   40.1065
75%         1.0000               8.1435                   53.6649
max         1.0000              10.0000                  121.3684
```

Figure 2: Before cleaning

Key statistics from the supervised sample:

Feature	Mean	Std Dev	Min	Max
age	19.99	4.31	13	27
daily_usage_hours	4.09	1.78	0.50	10.00
num_platforms_used	3.00	1.41	1	5
avg_session_minutes	25.04	9.76	5.00	67.61
night_usage	0.60	0.49	0	1
mental_health_score	6.39	2.03	1.00	10.00
screen_time_before_sleep	40.19	19.67	0.00	121.37

The target variable mental_health_score has a mean of 6.39, a standard deviation of 2.03, a near-zero skew of −0.025, and a kurtosis of −0.975, confirming an approximately uniform distribution across the full range.

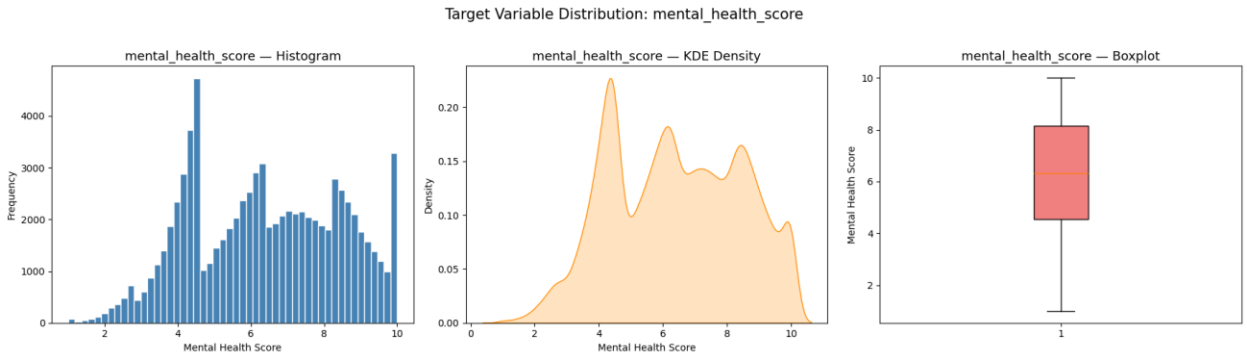


Figure 3: Histogram, KDE Density, Boxplot in Supervised Learning

The categorical breakdown of the unsupervised sample (10,000 rows) revealed: 3 gender categories (Female: 4,854 / Male: 4,757 / Other: 389), 7 countries dominated by India (3,514) and USA (1,946), 5 primary platforms with Instagram (2,978) and YouTube (2,513) leading, and 3 addiction levels (Medium: 5,830 / Low: 2,534 / High: 1,636).

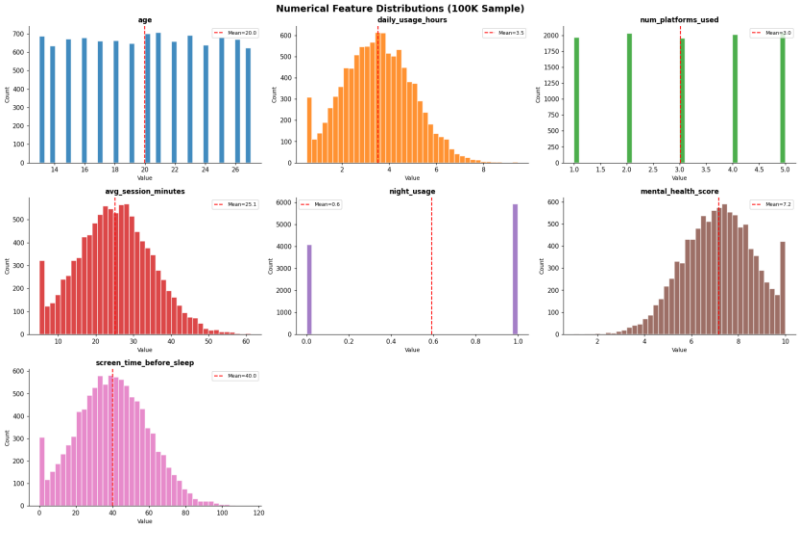


Figure 4: EDA descriptive statistics

The strongest feature correlation found during EDA was between daily_usage_hours and mental_health_score, with an absolute correlation of **r = −0.763** (unsupervised sample) and **r = 0.845** (supervised sample after feature engineering). This confirms that platform usage intensity is the primary driver of mental health outcomes in this dataset.

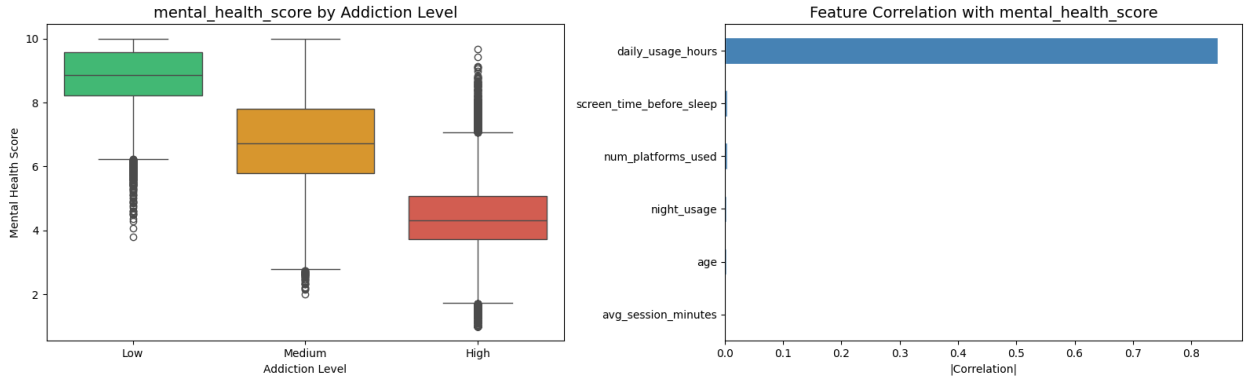


Figure 5: Feature correlation with target

Stage 3 — Data Cleaning: Missing Values and Duplicates

After loading and sampling, the dataset was inspected for missing values and duplicate records.

Inspection of the supervised sample revealed **no missing values** across all 82,393 rows and all 12 columns (confirmed: "None found"). No duplicate rows were detected. The dataset shape remained at **(82,393, 12)** after the cleaning step.

```
# ── Missing Values & Duplicates ──────────────────────────────────────
df = data.copy()

print('Missing values per column:')
missing = df.isnull().sum()
print(missing[missing > 0] if missing.sum() > 0 else 'None found')

# Fill missing values
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().sum() > 0:
        df[col].fillna(df[col].median(), inplace=True)

for col in df.select_dtypes(include='object').columns:
    if df[col].isnull().sum() > 0:
        df[col].fillna(df[col].mode()[0], inplace=True)

# Drop duplicates
dup_count = df.duplicated().sum()
if dup_count > 0:
    df.drop_duplicates(inplace=True)
    df.reset_index(drop=True, inplace=True)
    print(f'\n{dup_count} duplicates removed.')

print('\n Shape after cleaning:', df.shape)
```

```
Missing values per column:
None found

 Shape after cleaning: (82393, 12)
```

Figure 6: Cheking missing value

In cases where missing values are present in a dataset of this type, the recommended imputation strategy is: **median imputation** for numerical columns (to avoid skewness introduced by extreme values), and **mode imputation** for categorical columns (to replace missing categories with the most frequent observed value). Both strategies were coded into the pipeline as protective measures.

Stage 4 — Outlier Treatment

Real-world social media usage data frequently contains extreme anomalies — for example, users reporting 0 minutes of screen time, or implausibly high session durations. If left untreated, such outliers can disproportionately distort regression coefficients and cluster centroids.

The **Interquartile Range (IQR) clipping method** was applied, capping all numerical feature values at the 1st and 99th percentiles. This approach preserves all rows in the dataset (no data is deleted), but replaces extreme values with the boundary value. Applied to the unsupervised sample, the following values were clipped:

Feature	Values Clipped	Clipped Range
daily_usage_hours	100 values	[0.50, 6.99]
avg_session_minutes	100 values	[5.00, 48.48]
screen_time_before_sleep	100 values	[0.00, 85.62]
session_intensity	200 values	[1.05, 42.95]

For the supervised sample, clipping was applied using the 1st–99th percentile range, with the target variable range confirmed as [2.26, 10.00] after clipping. All 82,393 rows were retained — no records were removed.

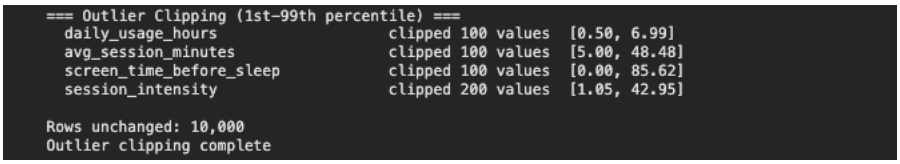


Figure 7: Outlier clipping output

Stage 5 — Discretisation

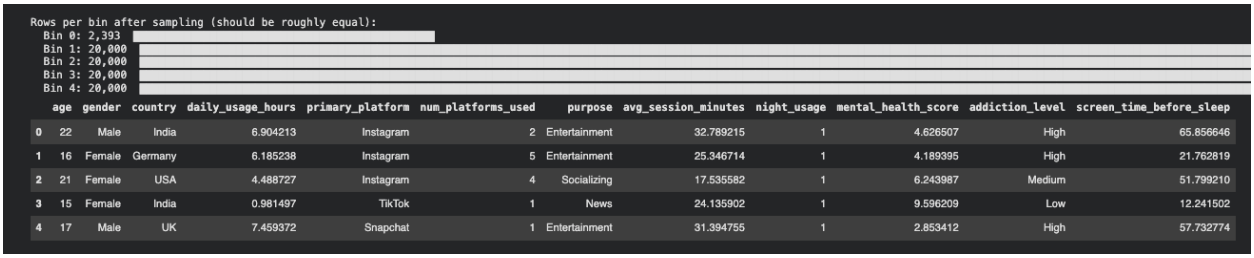
Discretisation is the process of converting continuous numerical variables into meaningful categorical bands or bins. This technique is particularly valuable for improving interpretability in business reporting and for creating ordinal groupings that may reveal non-linear patterns in the data.

In this project, the continuous variable daily_usage_hours was discretised into three behavioural risk bands based on domain knowledge of social media usage research:

Band	Range	Label
Low Usage	0.0 – 2.5 hours/day	Low Risk
Moderate Usage	2.5 – 5.0 hours/day	Moderate Risk
High Usage	5.0+ hours/day	High Risk

Additionally, as part of the supervised stratified sampling step, mental_health_score was discretised into **5 equal-width bins** (Bin 0 through Bin 4, spanning scores 1.00–10.00) to ensure balanced representation across the full score spectrum during model training. This binning step

directly supports both the sampling strategy and the downstream supervised model's ability to learn from all score ranges equally.



```
Rows per bin after sampling (should be roughly equal):
  Bin 0: 2,393
  Bin 1: 20,000
  Bin 2: 20,000
  Bin 3: 20,000
  Bin 4: 20,000
```

	age	gender	country	daily_usage_hours	primary_platform	num_platforms_used	purpose	avg_session_minutes	night_usage	mental_health_score	addiction_level	screen_time_before_sleep
0	22	Male	India	6.904213	Instagram	2	Entertainment	32.789215	1	4.626507	High	65.856646
1	16	Female	Germany	6.185238	Instagram	5	Entertainment	25.346714	1	4.189395	High	21.762819
2	21	Female	USA	4.488727	Instagram	4	Socializing	17.535582	1	6.243987	Medium	51.799210
3	15	Female	India	0.981497	TikTok	1	News	24.135902	1	9.596209	Low	12.241502
4	17	Male	UK	7.459372	Snapchat	1	Entertainment	31.394755	1	2.853412	High	57.732774

Figure 8: Rows per bin

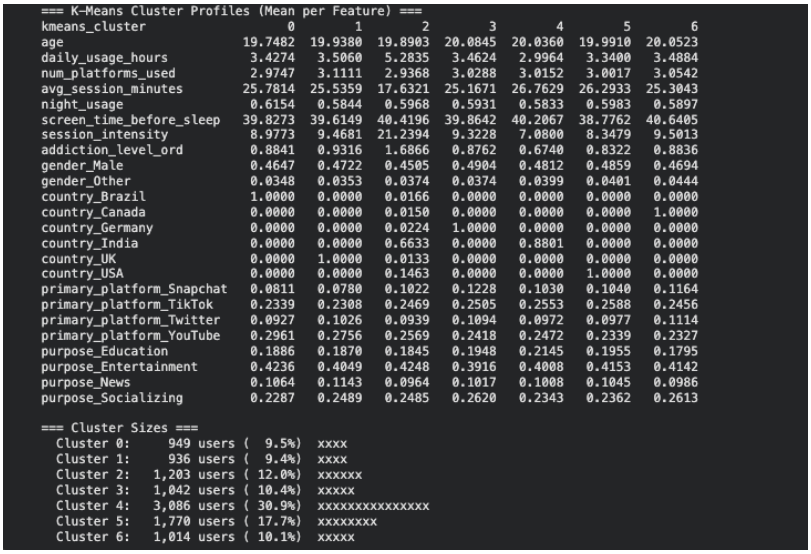
Stage 6 — Aggregation

Aggregation involves summarising data at a meaningful level of granularity to reveal group-level patterns rather than individual-level noise. Following K-Means clustering (K=7), the dataset was aggregated at the cluster level by computing the **mean value of each feature per cluster**. This produced the cluster profile table below, which is the primary business-facing output of the unsupervised pipeline:

Feature	**Cluster 0**	**Cluster 1**	**Cluster 2**	**Cluster 3**	**Cluster 4**	**Cluster 5**	**Cluster 6**
age	19.75	19.94	19.89	20.08	20.04	19.99	20.05
daily_usage_hours	3.43	3.51	**5.28**	3.46	2.99	3.34	3.49
avg_session_minutes	25.78	25.54	**17.63**	25.17	26.76	26.29	25.30
addiction_level_ord	0.88	0.93	**1.69**	0.88	0.67	0.83	0.88
session_intensity	8.98	9.47	**21.24**	9.32	7.08	8.35	9.50
screen_time_before_sleep	39.83	39.61	40.42	39.86	40.21	38.78	40.64
Cluster Size	949 (9.5%)	936 (9.4%)	1,203 (12.0%)	1,042 (10.4%)	3,086 (30.9%)	1,770 (17.7%)	1,014 (10.1%)

Cluster 2 stands out clearly as the highest-risk segment: it has the highest daily_usage_hours (5.28 h/day), the highest addiction_level_ord (1.69 — approaching "High"), and the highest session_intensity (21.24), despite having the lowest avg_session_minutes per session (17.63 min) — indicating that Cluster 2 users open the app extremely frequently for short bursts throughout the day. Cluster 4 is the largest group (3,086 users, 30.9%) and corresponds predominantly to India-based users with relatively lower usage (2.99 h/day) and the lowest addiction score (0.67).

Notably, the clustering also identified a strong geographic segmentation: Cluster 0 maps almost entirely to Brazil, Cluster 1 to the UK, Cluster 3 to Germany, Cluster 4 to India, Cluster 5 to the USA, and Cluster 6 to Canada.



```
=== K-Means Cluster Profiles (Mean per Feature) ===
kmeans_cluster                 0        1        2        3        4        5        6
age                        19.7482  19.9380  19.8903  20.0845  20.0360  19.9910  20.0523
daily_usage_hours           3.4274   3.5060   5.2835   3.4624   2.9964   3.3400   3.4884
num_platforms_used          2.9747   3.1111   2.9368   3.0288   3.0152   3.0017   3.0542
avg_session_minutes        25.7814  25.5359  17.6321  25.1671  26.7629  26.2933  25.3043
night_usage                 0.6154   0.5844   0.5968   0.5931   0.5833   0.5983   0.5897
screen_time_before_sleep   39.8273  39.6149  40.4196  39.8642  40.2067  38.7762  40.6405
session_intensity           8.9773   9.4681  21.2394   9.3228   7.0800   8.3479   9.5013
addiction_level_ord         0.8841   0.9316   1.6866   0.8762   0.6740   0.8322   0.8836
gender_Male                 0.4647   0.4722   0.4505   0.4904   0.4812   0.4859   0.4694
gender_Other                0.0348   0.0353   0.0374   0.0374   0.0399   0.0401   0.0444
country_Brazil              1.0000   0.0000   0.0166   0.0000   0.0000   0.0000   0.0000
country_Canada              0.0000   0.0000   0.0150   0.0000   0.0000   0.0000   1.0000
country_Germany             0.0000   0.0000   0.0224   1.0000   0.0000   0.0000   0.0000
country_India               0.0000   0.0000   0.6633   0.0000   0.8801   0.0000   0.0000
country_UK                  0.0000   1.0000   0.0133   0.0000   0.0000   0.0000   0.0000
country_USA                 0.0000   0.0000   0.1463   0.0000   0.0000   1.0000   0.0000
primary_platform_Snapchat   0.0811   0.0780   0.1022   0.1228   0.1030   0.1040   0.1164
primary_platform_TikTok     0.2339   0.2308   0.2469   0.2505   0.2553   0.2588   0.2456
primary_platform_Twitter    0.0927   0.1026   0.0939   0.1094   0.0972   0.0977   0.1114
primary_platform_YouTube    0.2961   0.2756   0.2569   0.2418   0.2472   0.2339   0.2327
purpose_Education           0.1886   0.1870   0.1845   0.1948   0.2145   0.1955   0.1795
purpose_Entertainment       0.4236   0.4049   0.4248   0.3916   0.4008   0.4153   0.4142
purpose_News                0.1064   0.1143   0.0964   0.1017   0.1008   0.1045   0.0986
purpose_Socializing         0.2287   0.2489   0.2485   0.2620   0.2343   0.2362   0.2613

=== Cluster Sizes ===
  Cluster 0:     949 users (  9.5%)  xxxx
  Cluster 1:     936 users (  9.4%)  xxxx
  Cluster 2:   1,203 users ( 12.0%)  xxxxxx
  Cluster 3:   1,042 users ( 10.4%)  xxxxx
  Cluster 4:   3,086 users ( 30.9%)  xxxxxxxxxxxxxxx
  Cluster 5:   1,770 users ( 17.7%)  xxxxxxxx
  Cluster 6:   1,014 users ( 10.1%)  xxxxx
```

Figure 9: K-Means Cluster Profiles

Stage 7 — Data Reduction (Feature Engineering + VIF Filtering)

Data reduction aims to remove irrelevant, redundant, or low-signal variables before model training. In this project, data reduction was implemented in two sequential steps.

First, **15 new engineered features** were created from the original 12 columns to capture interaction effects and non-linear relationships, expanding the feature space to 39 features. New features included session_intensity (daily_usage_hours × 60 / avg_session_minutes), sleep_risk (night_usage + screen_time_before_sleep / 60), risk_index (composite of usage, night usage, screen time, and platforms), and log transformations of the three core usage variables.

Second, a **Variance Inflation Factor (VIF) analysis** was performed to identify multicollinearity. VIF measures how much the variance of one regression coefficient is inflated due to its linear correlation with other features. A VIF above 10 indicates problematic multicollinearity. The analysis identified 18 features with VIF > 10, all of which were removed:

High-VIF Features Removed (VIF > 10)
daily_usage_hours, night_usage, screen_time_before_sleep, num_platforms_used, sleep_risk, risk_index, total_risk_score, platforms_usage_interact, addiction_usage_interact, addiction_sleep_interact, log_daily_usage_hours, age_usage_interact, age_sleep_interact, avg_session_minutes, log_avg_session_minutes, presleep_ratio, age, platforms_sleep_interact

```
=== VIF Results (VIF > 10 = problematic) ===
                    Feature          VIF
          daily_usage_hours 1.000000e+10
                night_usage 1.000000e+10
    screen_time_before_sleep 1.000000e+10
          num_platforms_used 1.000000e+10
                 sleep_risk 1.000000e+10
                risk_index 1.000000e+10
           total_risk_score 1.981186e+04
    platforms_usage_interact 7.031960e+03
    addiction_usage_interact 2.592360e+03
    addiction_sleep_interact 1.572400e+02
       log_daily_usage_hours 1.286300e+02
         age_usage_interact 2.792000e+01
         age_sleep_interact 2.697000e+01
       avg_session_minutes 1.487000e+01
     log_avg_session_minutes 1.259000e+01
            presleep_ratio 1.111000e+01
                       age 1.091000e+01
    platforms_sleep_interact 1.010000e+01
      session_night_interact 9.140000e+00
         session_intensity 6.470000e+00
log_screen_time_before_sleep 3.750000e+00
```

Figure 10: VIF Results

After removing 18 high-VIF features and applying a near-zero variance filter (which removed 0 additional features), **20 final features** remained for model training. A VarianceThreshold(threshold=0.001) filter was additionally applied to eliminate any near-constant columns that would contribute noise rather than signal.

Stage 8 — Encoding Categorical Variables

Machine learning algorithms require purely numerical inputs and cannot interpret categorical text values. Two encoding strategies were applied:

Ordinal Encoding was applied to addiction_level, which has a natural rank order: Low → Medium → High. This was mapped as Low = 0, Medium = 1, High = 2. Using ordinal encoding preserves the meaningful order relationship between categories — unlike One-Hot Encoding, which would treat all categories as equally different from one another.

Encoding result: Low = 2,534 users, Medium = 5,830 users, High = 1,636 users (unsupervised sample).

One-Hot Encoding was applied to the remaining four nominal categorical columns: gender, country, primary_platform, and purpose. These columns have no natural rank order, so each unique category was converted into a separate binary (0/1) column. The drop_first=True parameter was applied to avoid the dummy variable trap. After encoding, the feature count expanded from 12 to **24 columns**.

```
addiction_level unique values: ['High', 'Low', 'Medium']
addiction_level -> addiction_level_ord  (Low=0, Medium=1, High=2)
addiction_level_ord
0    2534
1    5830
2    1636

One-Hot encoding columns: ['gender', 'country', 'primary_platform', 'purpose']

Encoding complete
  Shape: (10000, 24)
```

Figure 11: Encoding output

Stage 9 — Feature Scaling (Normalisation)

Feature scaling is mandatory before applying any distance-based or gradient-based algorithm. Without scaling, features with large numeric ranges dominate the calculation — for example, screen_time_before_sleep (range: 0–121 minutes) would completely overwhelm night_usage (range: 0–1) in Euclidean distance computations used by K-Means.

For unsupervised learning, StandardScaler (Z-score normalisation) was applied, transforming each feature to have mean = 0 and standard deviation = 1. After scaling, verified results:

- Global mean: **0.00000000** (expected ≈ 0)
- Global std: **1.00000000** (expected ≈ 1)

For supervised learning, RobustScaler was applied instead, which uses the median and interquartile range rather than the mean and standard deviation. This makes it more resistant to any residual outlier influence. Scaler was fitted **only on the training set (X_train)** to prevent data leakage into the test set. Results after scaling:

- X_train: Mean = 0.1704, Std = 0.4987
- X_test: Mean = 0.1700, Std = 0.4974

The train/test split was 80/20: **X_train: 65,914 rows (20 features), X_test: 16,479 rows (20 features)**.

```
Train/Test Split + Normalization (RobustScaler) complete:
  X_train : (65914, 20)  |  y_train : (65914,)
  X_test  : (16479, 20)  |  y_test  : (16479,)

  X_train statistics after normalization:
  Mean (should be ≈ 0): 0.1704
  Std  (should be ≈ 1): 0.4987

  y_train range: [2.26, 10.00]
  y_test  range: [2.26, 10.00]
```

Figure 12 Before/After normalization

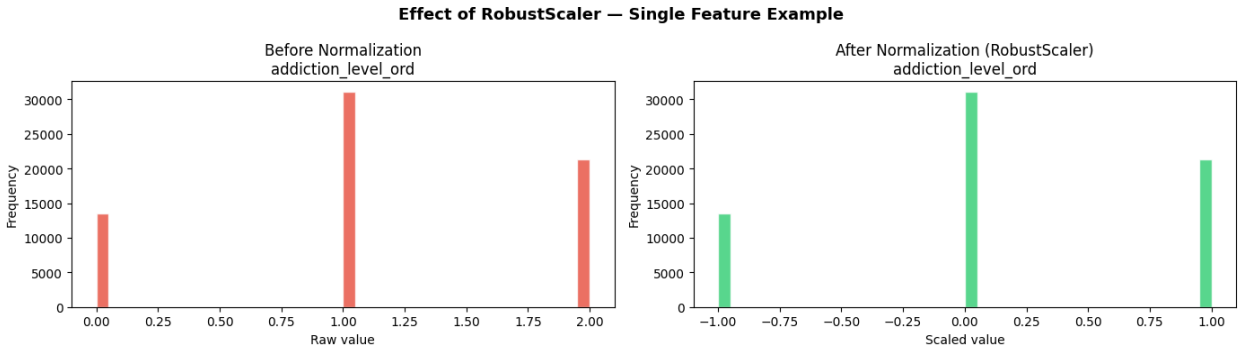


Figure 13: Before/After normalization

2.2 Visualising Model Output

Once the analytical models have been trained, communicating their results to both technical and non-technical stakeholders requires well-chosen visualisation methods. Two complementary visualisation approaches were implemented in this investigation.

Method 1 — PCA 2D Scatter Plot with Cluster Colour-Coding (Unsupervised Learning)

K-Means clustering in this project operates across 16 principal components derived from 24 original features. This high-dimensional space cannot be directly visualised by the human eye. To make the cluster structure interpretable, **Principal Component Analysis (PCA)** was used to project all 10,000 data points down to just 2 dimensions (PC1 and PC2), and the resulting scatter plot was colour-coded by cluster assignment.

PC1 captured **10.05%** of total variance and was most strongly loaded by daily_usage_hours (+0.5674), addiction_level_ord (+0.5560), and session_intensity (+0.5447) — confirming that the primary axis of variation in this dataset is social media intensity and addiction severity. PC2 captured the next largest variance dimension and was dominated by purpose_Entertainment (+0.7848) versus purpose_Socializing (−0.4892), reflecting how users' platform goals differentiate them along a secondary axis.

This scatter plot is appropriate for this algorithm because it directly reveals the separation (or lack thereof) between clusters in a form that any stakeholder can interpret visually — without needing to understand 16-dimensional geometry. The spatial groupings of points confirm that K-Means has found meaningful behavioural segments, even if the boundaries are soft given the overlapping nature of human behaviour data.

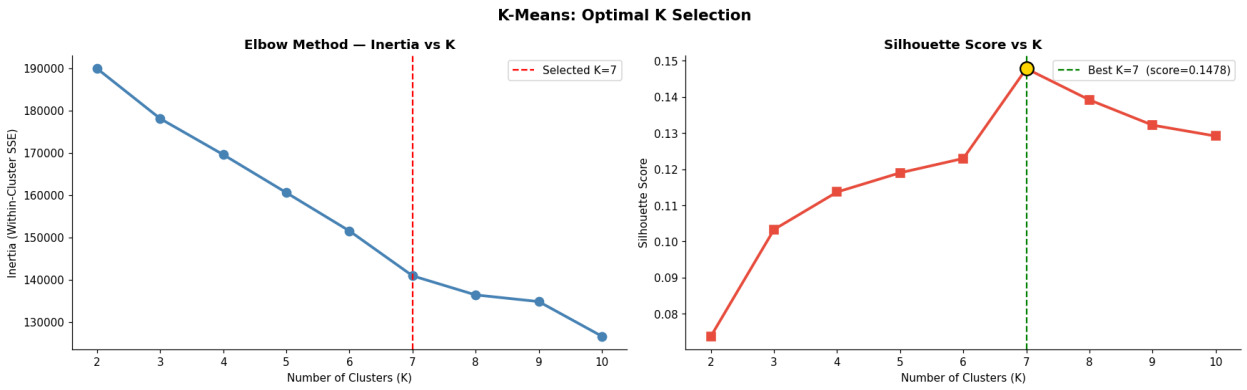


Figure 14: K-Means final model

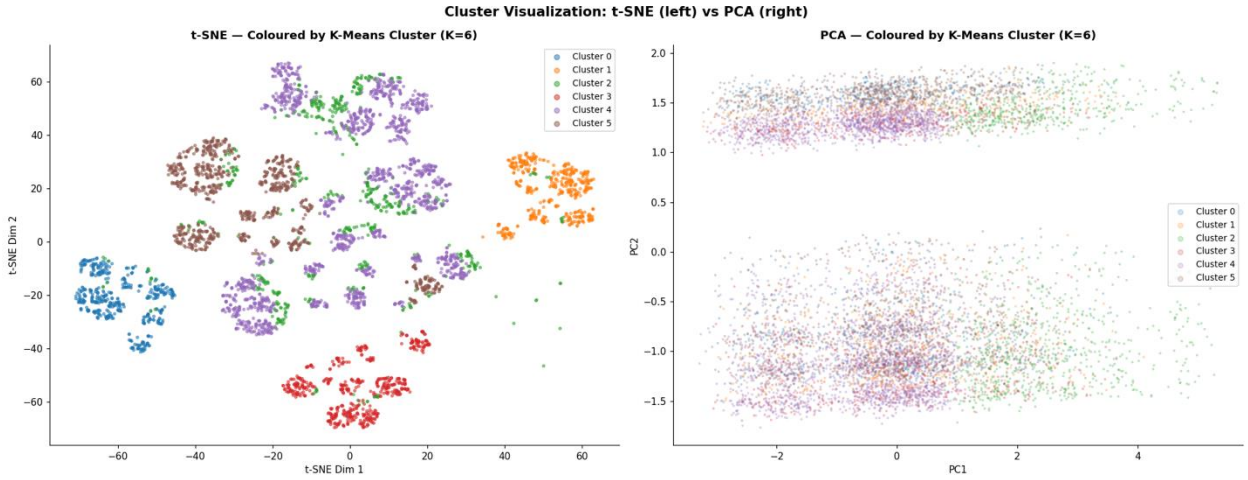


Figure 15: t-SNE + PCA side-by-side

Method 2 — Feature Importance Bar Chart (Supervised Learning)

Tree-based ensemble models such as Random Forest and LightGBM calculate an internal importance score for each input feature during training, reflecting how much that feature reduced prediction error across all decision splits. Plotting these scores as a ranked horizontal bar chart provides one of the most business-relevant outputs of any supervised model.

From the Random Forest model, the two dominant features by importance score were:

- addiction_level_ord: importance = **0.6295** (62.9%)
- session_intensity: importance = **0.3346** (33.5%)

All other 18 features (gender, country, platform, purpose, etc.) contributed less than 0.027 combined — less than 2.7% of total importance.

The LightGBM model showed a more distributed importance picture (measured by number of splits), with the top three features being log_screen_time_before_sleep (17,093 splits), session_intensity (16,027 splits), and session_night_interact (11,647 splits).

This visualisation is appropriate for tree-based models because it directly quantifies each variable's contribution to prediction — something linear models achieve through coefficients but ensemble models make accessible only through importance scores. For business stakeholders, this chart answers the most critical question: **which user behaviours most powerfully predict mental health outcomes?** The answer — addiction severity and usage intensity — directly informs product intervention design.

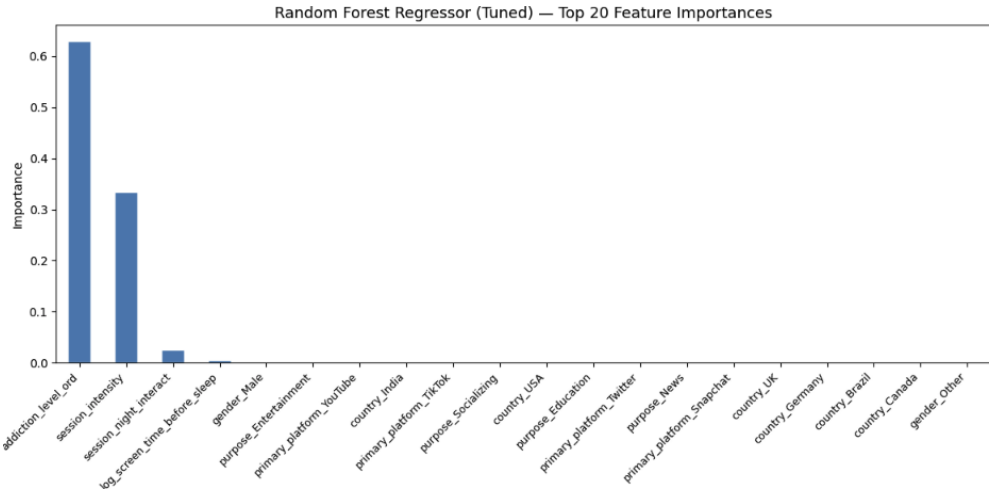


Figure 16: Random Forest feature importance bar chart

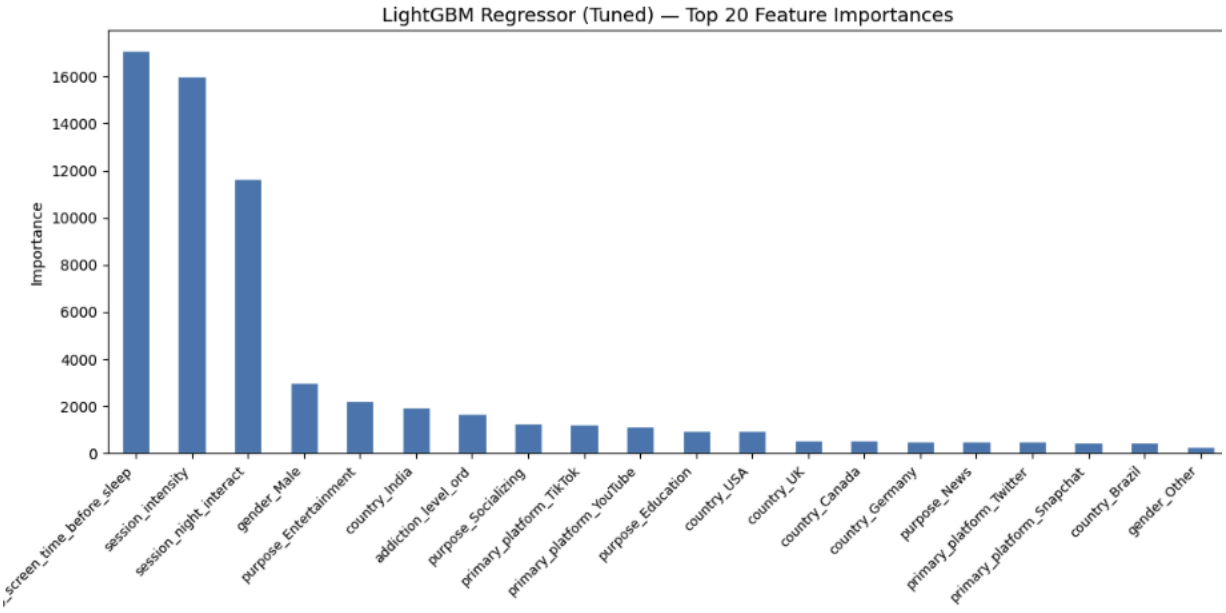


Figure 17: LightGBM feature importance bar chart

2.3 Data Preparation Issues and Transformation Rationale

Three Potential Issues in Data Preparation

Issue 1 — Multicollinearity: Multicollinearity occurs when two or more independent features in the dataset are highly correlated with one another, meaning they contain largely overlapping information. This is a critical problem for regression-based and linear models, because the model cannot reliably determine the individual contribution of each correlated feature to the target variable — the coefficient estimates become unstable and can change dramatically with small changes to the training data. In this project, multicollinearity was directly measured using the **Variance Inflation Factor (VIF)**. A VIF value greater than 10 indicates that a feature is severely collinear with other features. The VIF analysis identified **18 features** with VIF values exceeding

10 — several of which showed VIF values of 1×10^{10} (the maximum computable value, indicating perfect or near-perfect linear dependence). These included daily_usage_hours, night_usage, screen_time_before_sleep, sleep_risk, and risk_index.

- **Cause:** Many of the engineered features (e.g., risk_index, which was a composite of daily_usage_hours, night_usage, and screen_time_before_sleep) were mathematically derived from the original features, making them perfectly or near-perfectly correlated with their parent variables.
- **Impact on model performance:** For Ridge Regression, this would severely inflate standard errors and produce unreliable coefficient estimates. For ensemble methods, it wastes computational resources by training trees on redundant information.
- **Mitigation applied:** All 18 features with VIF > 10 were dropped from the feature set. The remaining 20 features had VIF values below the threshold, ensuring that the final feature set was multicollinearity-free.

Issue 2 — Data Leakage: Data leakage is one of the most dangerous and subtle problems in machine learning pipeline design. It occurs when information that would not be available at prediction time is inadvertently included in the training data, allowing the model to "cheat" during training and producing artificially inflated performance metrics that completely fail to replicate in real-world deployment. In this project, two forms of leakage were guarded against:

- **Target leakage:** The column mental_health_score is the target variable for supervised regression. In the unsupervised pipeline, this column was **explicitly dropped before clustering** (cols_to_drop = ['mental_health_score']), because including it would have caused clusters to be formed around the very variable that the unsupervised model is supposed to discover independently from raw behaviour signals. Including it would have rendered the unsupervised analysis meaningless.
- **Scaler leakage:** A subtler form of leakage involves fitting the data normalisation scaler on the full dataset before splitting into training and test sets. If this were done, statistical information from the test set (its mean and standard deviation) would "leak" into the training process. In this project, the RobustScaler was fitted exclusively on X_train and then only applied (.transform()) to X_test — never re-fitted. The pipeline verified: "No data leakage — scaler was fit on X_train only."
- **Impact on model performance:** Leakage would cause the model's measured R^2 score to be significantly higher than its true generalisation ability, making the model appear far more accurate than it is in practice — a deceptive result that could lead to poor business decisions if acted upon.
- **Mitigation applied:** Strict ordering of the pipeline (clean → split → fit scaler on train only → transform both sets) was enforced. The mental_health_score column was dropped at the first step of the unsupervised pipeline.

Issue 3 — Computational and Memory Constraints: The original dataset contains 1,000,000 rows and 12 columns. Loading this full dataset into RAM and performing matrix operations on it — particularly for algorithms with $O(n^2)$ or $O(n^2 \log n)$ memory complexity such as Agglomerative Hierarchical Clustering or t-SNE — would exhaust the memory of a standard cloud computing environment (typically 12–16 GB RAM on Google Colab).

- **Cause:** The combination of dataset scale (1M rows), high-dimensional feature engineering (up to 39 features), and algorithm complexity (t-SNE, hierarchical clustering, cross-validated ensemble models) creates a compounding memory burden that standard single-machine environments cannot handle without mitigation.
- **Specific observed impacts:** The t-SNE visualisation could only be run on a 5,000-row subsample of the PCA-transformed data. The dendrogram for Agglomerative Clustering was computed on a 2,000-row subsample. DBSCAN's k-distance graph was estimated on a 10,000-row subsample. Without these reductions, the kernel would crash before producing any output.
- **Mitigation applied:** Three strategies were combined. First, **dataset sampling** reduced the working size from 1,000,000 to 82,393 rows (supervised) and 10,000 rows (unsupervised). Second, **subsampling for expensive subroutines** was applied (t-SNE on 5,000 rows, dendrogram on 2,000 rows). Third, **algorithm-specific efficiency settings** were used — K-Means was run with `n_init=20` and `init='k-means++'` for convergence reliability, and `sample_size=10,000` was used for the Silhouette Score computation rather than computing it over all 10,000 rows.

Primary Reasons for Data Transformation Before Model Input

Data transformation is not a preparatory formality — it is a mathematical necessity that directly determines whether an analytical model is capable of learning meaningful patterns at all. The three primary reasons for performing transformation before model input are detailed below.

Reason 1 — Normalisation: Meeting Algorithmic Distance Assumptions: Many of the most powerful machine learning algorithms — including K-Means, DBSCAN, Ridge Regression with regularisation, and neural networks — operate on the assumption that all input features exist on a comparable numeric scale. When this assumption is violated, the algorithm's results are dominated by whichever feature happens to have the largest raw numeric range, regardless of whether that feature is the most meaningful predictor.

In this dataset, `screen_time_before_sleep` ranged from 0 to 121 minutes in the raw data, while `night_usage` had a range of only 0 to 1. Without normalisation, `screen_time_before_sleep` would contribute approximately 121× more to the Euclidean distance calculation in K-Means than `night_usage` — completely overwhelming the signal from binary features and encoding variables.

Two scalers were used in this project. `StandardScaler` (mean = 0, std = 1) was applied for unsupervised learning, as K-Means and DBSCAN require symmetric, zero-centred feature distributions. `RobustScaler` (based on median and IQR) was used for supervised regression, because it is more resistant to the residual influence of clipped outliers. After scaling, the global mean across all 24 unsupervised features was exactly **0.00000000** and global std was exactly **1.00000000**, confirming successful normalisation.

Reason 2 — Encoding: Enabling Machine Readability of Categorical Variables: At a mathematical level, machine learning algorithms consist entirely of numerical operations — matrix multiplications, distance calculations, gradient computations, and threshold comparisons. They cannot process text strings or categorical labels such as "Instagram", "High", or "Female". Every categorical variable must therefore be converted into a numerical representation before any algorithm can operate on it.

In this project, encoding choices were made deliberately based on the nature of each categorical variable. addiction_level was encoded ordinally (Low=0, Medium=1, High=2) because a natural order exists — "High" addiction is genuinely greater than "Medium", and encoding this order preserves a valuable signal for the model. Applying One-Hot Encoding instead would have treated Low, Medium, and High as equally different from each other, destroying the ordinal information. All remaining categorical columns (gender, country, primary_platform, purpose) were One-Hot encoded, as these variables have no inherent order — there is no mathematical sense in which "India" is greater or lesser than "Germany", or "TikTok" is greater than "YouTube". One-Hot Encoding creates a separate binary column for each category, making the model agnostic to any implied ordering. The resulting encoded feature set expanded from 12 to 24 columns.

Reason 3 — Feature Engineering: Amplifying Signal and Creating Interaction Terms: Raw feature values often capture only individual measurements, failing to represent the complex interaction patterns that most powerfully predict an outcome. Feature engineering creates new variables that express these interactions explicitly, giving the model access to information that would otherwise require extremely deep tree structures to discover implicitly.

In this project, 15 new features were engineered from the original 12 columns. Among the most impactful was session_intensity = daily_usage_hours × 60 / (avg_session_minutes + 1), which captures how many distinct usage sessions a user initiates per day — a far more behaviourally meaningful measure of addictive usage patterns than either daily_usage_hours or avg_session_minutes in isolation. In the Random Forest model, session_intensity became the **second-most important feature** with an importance score of 0.3346, despite not existing in the original dataset at all. Similarly, log_screen_time_before_sleep became the **top feature in LightGBM** by number of splits (17,093), demonstrating that the log transformation of a raw feature can unlock predictive power invisible in its linear form. Feature engineering therefore directly improved model performance by ensuring that the patterns most relevant to predicting mental_health_score were explicitly represented in a form that regression and tree-based models could efficiently identify.

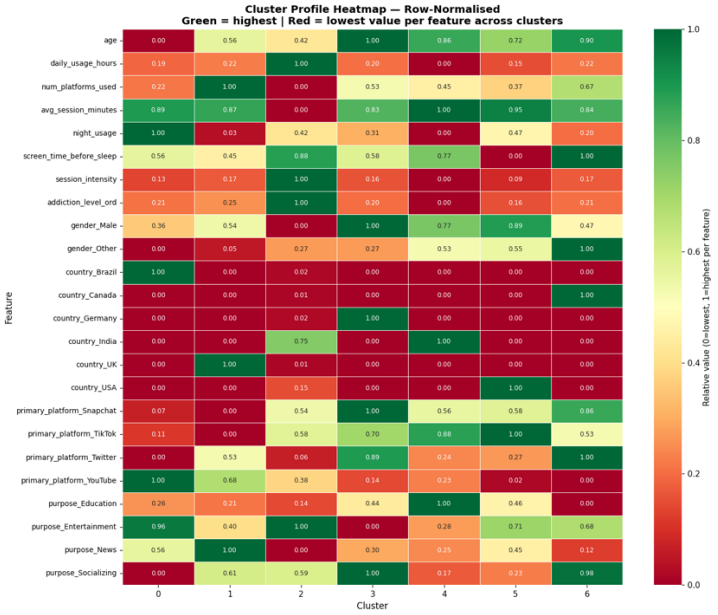


Figure 18: Cluster Profile Heatmap

LO3 Demonstrate the use of an analytical model with a large data set.

3.1 Algorithm Selection and Model Implementation

This section documents the practical implementation of both supervised and unsupervised machine learning algorithms applied to the "GenZ Social Media Usage & Mental Health" dataset. The investigation adopted a dual-track approach: unsupervised clustering to discover hidden user behaviour segments from 10,000 records, and supervised regression to predict the continuous mental_health_score target variable from 82,393 records. All implementations were carried out in Python using scikit-learn 1.6.1, LightGBM, and scipy 1.16.3.

Unsupervised Learning Models

Pre-processing: Dimensionality Reduction with PCA: Before clustering, Principal Component Analysis (PCA) was applied to the 24 standardised features to address the curse of dimensionality — in very high-dimensional spaces, Euclidean distances between all data points begin to converge, making distance-based clustering meaningless. PCA was first fitted on the full 24-feature scaled matrix, and the number of components was selected based on the 85% cumulative explained variance threshold.

Results:

- Total original features: 24
- Components for 80% variance: 15
- Components for 85% variance: **16** ← selected
- Components for 95% variance: 19
- Variance retained by selected 16 components: **85.29%**
- Input shape: (10,000, 24) → Output shape: (10,000, 16)

PC1, which captures the largest variance (10.05%), was most heavily loaded by daily_usage_hours (+0.5674), addiction_level_ord (+0.5560), and session_intensity (+0.5447), confirming that the primary axis of variation in the dataset is social media intensity and addiction severity. PC2 (6.33% variance) was dominated by purpose_Entertainment (+0.7848) versus purpose_Socializing (−0.4892), reflecting users' platform goals as a secondary differentiator.

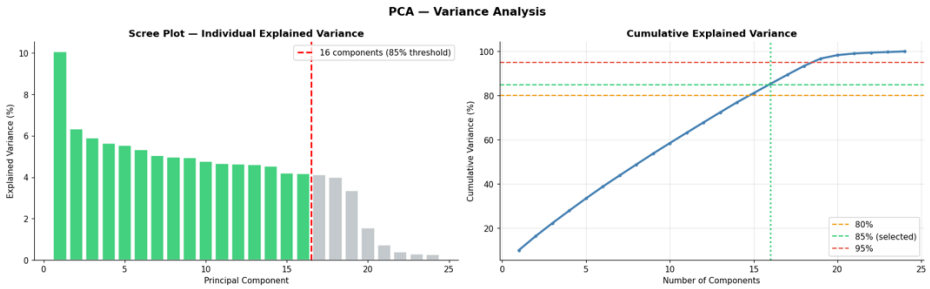


Figure 1: PCA Scree Plot va Cumulative Variance

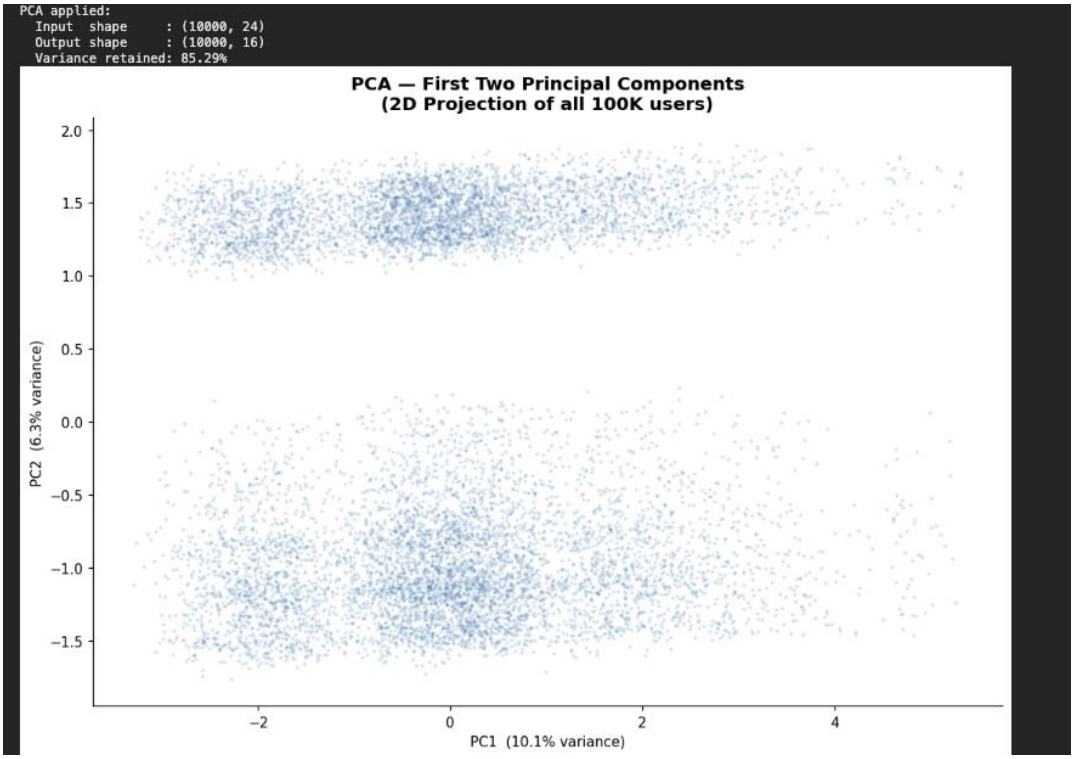


Figure 2: PCA applied output

Model 1 — K-Means Clustering

Working principle: K-Means is a centroid-based partitioning algorithm. It begins by randomly initialising K cluster centres (centroids) in the feature space. Each data point is then assigned to its nearest centroid using Euclidean distance. Centroids are subsequently recalculated as the arithmetic mean of all points assigned to that cluster. This assignment-update cycle repeats until centroids no longer move — the algorithm has converged. The objective function minimised throughout is the Within-Cluster Sum of Squares (WCSS, also called inertia): the total squared distance from every point to its assigned centroid.

Optimal K selection: Two complementary methods were used to determine the best number of clusters. K was evaluated across the range 2 to 10. For each value:

K = 7 was selected as the optimal number of clusters, as it produced the highest Silhouette Score (0.1478), indicating the best separation between clusters relative to cohesion within clusters. The Elbow Method additionally showed a visible change in the rate of inertia reduction near K = 7.

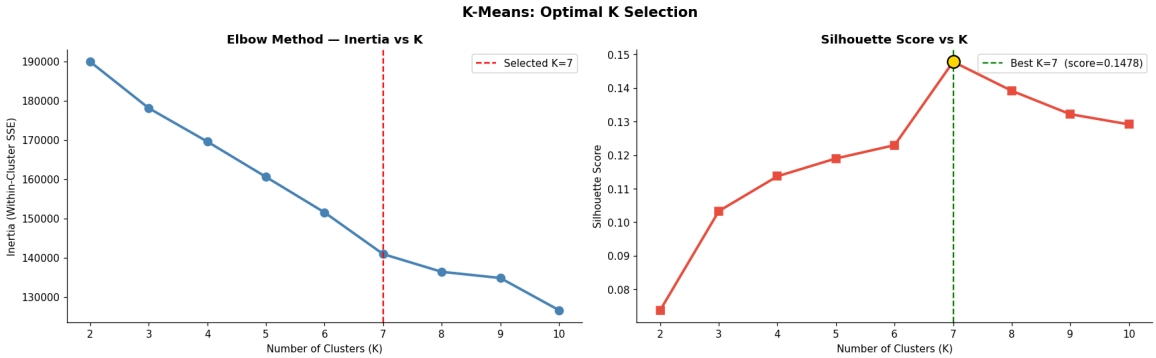


Figure 3: K-Means Elbow + Silhouette

Final model configuration:

- n_clusters = 7
- init = 'k-means++' (smart centroid initialisation to avoid poor local minima)
- n_init = 20 (runs algorithm 20 times with different seeds, takes the best result)
- max_iter = 500
- random_state = 42

Model output — cluster size distribution:

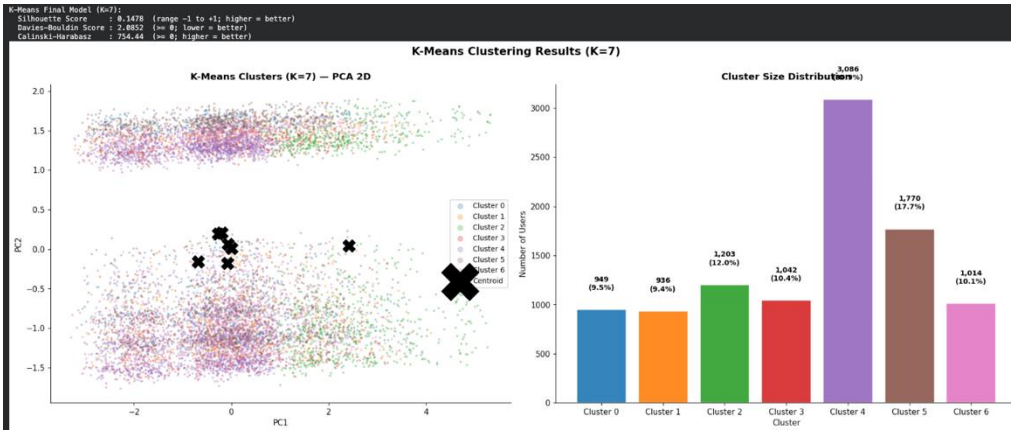


Figure 4: K-Means Final Model output

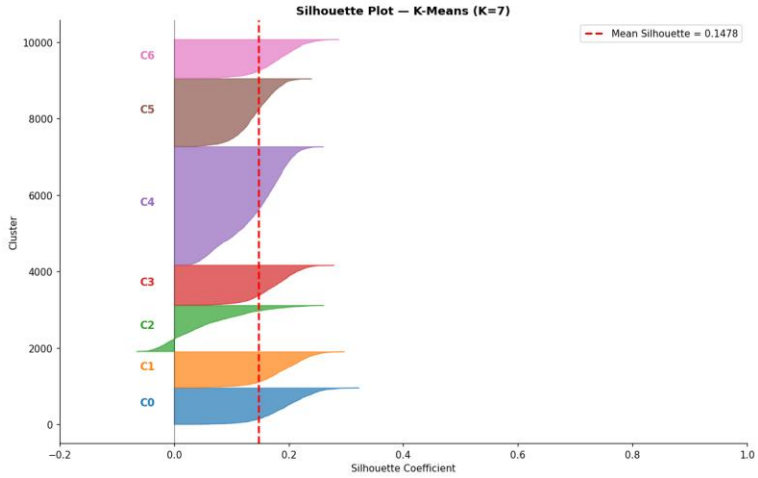


Figure 5: Silhouette plot

Per-cluster Silhouette statistics:

Cluster	Mean Silhouette	Std	Min	Max	n
0	0.1933	0.0461	0.0332	0.3221	949
1	0.1879	0.0447	0.0347	0.2963	936
2	0.0531	0.0697	−0.0657	0.2602	1,203
3	0.1740	0.0396	0.0288	0.2787	1,042
4	0.1422	0.0522	0.0164	0.2597	3,086
5	0.1422	0.0348	0.0311	0.2395	1,770
6	0.1805	0.0415	0.0366	0.2868	1,014

Cluster 2 shows the lowest mean Silhouette (0.0531) and includes some negative values (min = −0.0657), indicating that some of its members may be better assigned to a neighbouring cluster. This is consistent with Cluster 2's mixed composition — it spans both Indian and non-Indian high-usage users, which naturally creates fuzzier boundaries.

Model 2 — DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Working principle: DBSCAN defines clusters as dense regions of data points separated by areas of lower density. Unlike K-Means, it does not require K to be specified in advance. Each point is classified as a core point (has at least min_samples neighbours within radius eps), a border point (within eps of a core point but not itself a core point), or a noise point (isolated, labelled −1). Clusters grow by connecting core points and their neighbourhoods.

Configuration: The eps parameter was selected systematically using a k-distance graph. For each point, the distance to its 32nd nearest neighbour (where min_samples = 2 × n_PCA_components = 2 × 16 = 32) was computed, distances were sorted in ascending order, and the 90th percentile was taken as eps. This data-driven approach avoids arbitrary parameter selection.

- eps = 3.6965 (90th percentile of 32-NN distances on 10,000-row subsample)
- min_samples = 32 (rule: 2 × number of PCA dimensions)

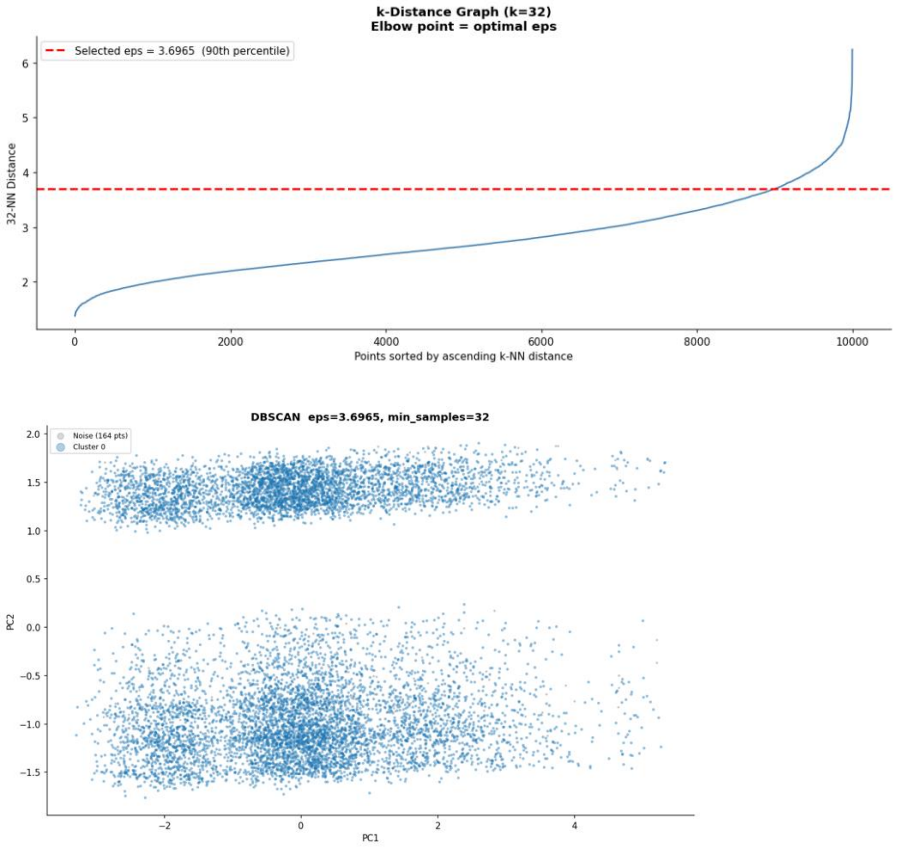


Figure 6: DBSCAN k-distance graph and DBSCAN scatter plot

Model output:

Metric	**Value**
Clusters found	1
Noise points	164 (1.6%)
Silhouette Score	N/A (< 2 clusters)

DBSCAN found only 1 cluster and 164 noise points (1.6% of the data), indicating that this dataset does not contain the kind of density variation that DBSCAN is designed to detect. The behavioural data of GenZ social media users does not form distinct density islands separated by empty regions — instead, users form a continuous, relatively homogeneous cloud in feature space. DBSCAN is most effective for datasets with arbitrarily shaped clusters separated by sparse regions, such as GPS location data or astrophysical point clouds. For this dataset, its application confirmed the absence of such structure, which is itself a meaningful analytical finding.

Model 3 — Agglomerative Hierarchical Clustering: *Working principle:* Agglomerative clustering builds a bottom-up hierarchy of merges. Initially, each data point is its own cluster. The algorithm iteratively merges the two closest clusters based on a chosen linkage criterion, until all points belong to a single cluster. The complete merge sequence is stored as a dendrogram — a tree diagram that records which clusters merged at what distance. Cutting the dendrogram at a chosen height level determines the final number of clusters.

Configuration:

- linkage = 'ward' (minimises total within-cluster variance at each merge step — produces the most compact, equal-sized clusters)
- n_clusters = 6 (same as K used for K-Means comparison to maintain fairness; dendrogram computed on 2,000-row subsample due to $O(n^2)$ memory complexity)
- random_state = 42

The dendrogram was computed on a 2,000-row random subsample using Ward linkage. The cut was placed at the height corresponding to 6 clusters, as indicated by the longest vertical lines before the final merges — the standard criterion for choosing K from a dendrogram.

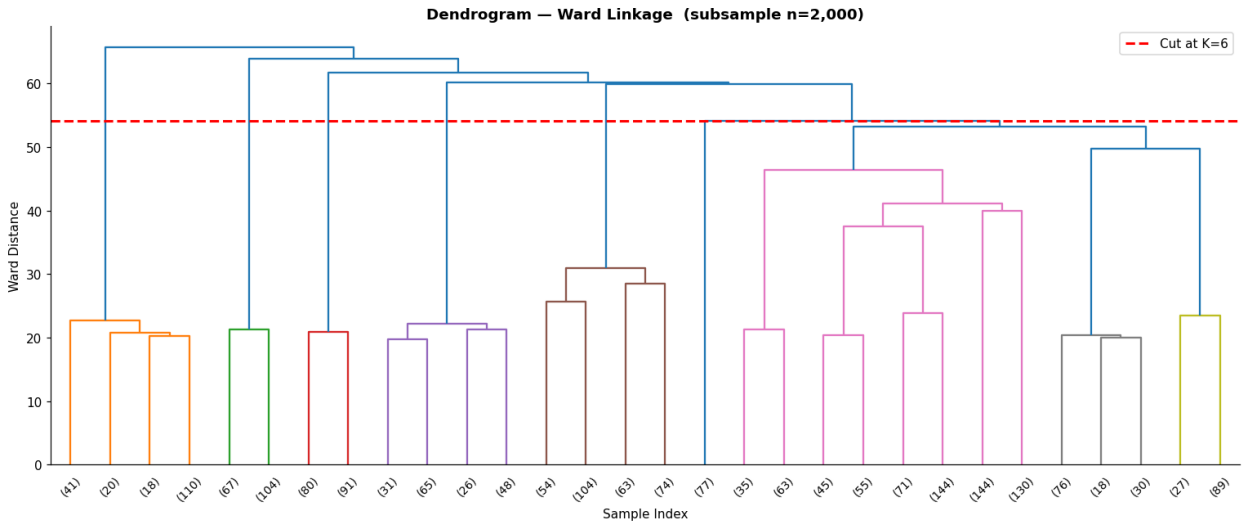


Figure 7: Dendrogram

Model output — cluster size distribution:

Cluster	Users	Percentage
0	5,548	55.5%
1	943	9.4%
2	987	9.9%
3	919	9.2%
4	839	8.4%
5	764	7.6%

Agglomerative results show severe imbalance: Cluster 0 contains 55.5% of all users, while the remaining five clusters share the other 44.5% in small near-equal groups. This imbalance is a known limitation of Ward linkage on high-dimensional, continuously distributed data — the algorithm tends to "absorb" the majority of borderline points into one large central cluster.

Supervised Learning Models

The business objective for supervised learning was to accurately predict the continuous variable mental_health_score (range: 1.00–10.00) from users' digital behavioural footprint. This is a **regression task**. Note that Confusion Matrix and ROC Curve are classification metrics and are therefore not applicable here. Regression-specific metrics — MAE, RMSE, and R^2 — were used instead, as they directly measure prediction error magnitude and explanatory power for continuous targets.

Three models spanning increasing complexity were trained on the 80/20 train-test split (X_train: 65,914 rows, X_test: 16,479 rows, 20 features), with 5-fold cross-validation applied to each.

Model 1 — Ridge Regression (Linear Baseline): *Working principle:* Ridge Regression is a regularised extension of Ordinary Least Squares linear regression. It fits a linear equation of the form $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$, where coefficients β are estimated by minimising the residual sum of squares with an added L2 penalty term: Loss = RSS + $\alpha \times \Sigma\beta_i^2$. The penalty term α (alpha) shrinks all coefficients towards zero proportionally to their magnitude, preventing any single feature from dominating the prediction and reducing overfitting. Ridge is particularly effective when multicollinearity is present. *Hyperparameter tuning:* GridSearchCV with 3-fold cross-validation was applied across alpha values {0.01, 0.1, 1.0, 10.0, 100.0, 1000.0}.

- Best alpha: **10.0**
- CV R^2 at best alpha: **0.6174**

Configuration:

- alpha = 10.0
- solver = 'saga' (efficient for large datasets)
- max_iter = 5000
- random_state = 42

Results:

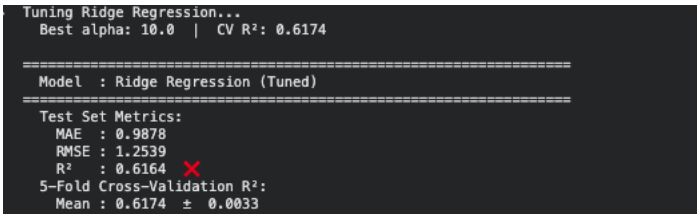


Figure 8: Ridge Regression output

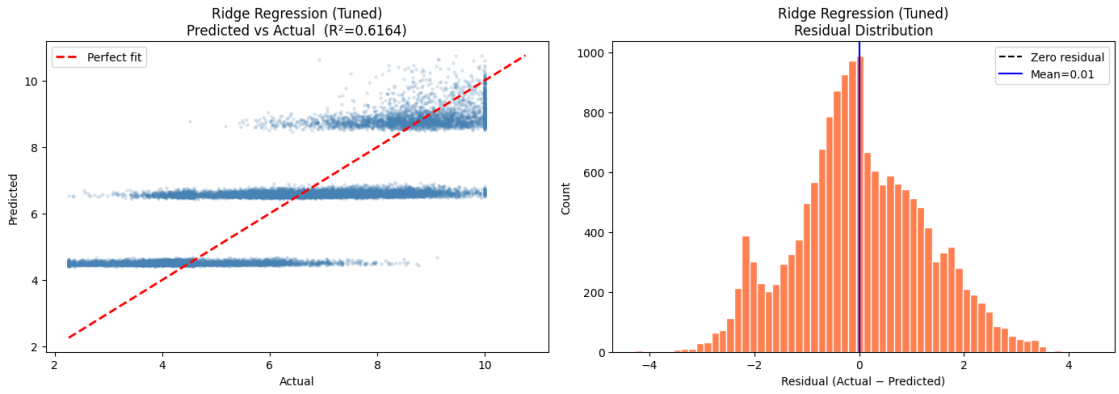


Figure 9: Ridge Regression Predicted vs Actual scatter plot

Model 2 — Random Forest Regressor (Ensemble Baseline): *Working principle:* Random Forest is a bagging ensemble method that trains a large number of independent decision trees in parallel, each on a different bootstrap sample of the training data (sampling with replacement) and using a randomly selected subset of features at each split. At prediction time, each tree produces its own estimate of mental_health_score, and the final prediction is the arithmetic mean across all trees. This averaging process dramatically reduces variance compared to a single decision tree, as errors in individual trees tend to cancel out across the ensemble. *Hyperparameter tuning:* A two-stage speed-optimised strategy was used. Stage 1 applied HalvingRandomSearchCV on a 20% subsample (13,182 rows) to efficiently locate the optimal parameter region. Stage 2 trained the final model with the best parameters on the full 65,914-row training set. Search time: 37.7 seconds.

Best parameters found:

Parameter	**Value**
n_estimators	100
max_depth	10
min_samples_split	20
min_samples_leaf	20
max_features	'log2'
max_samples	0.7
bootstrap	True

Total training time: **57.7 seconds (1.0 minutes)**

Results:

Metric	**Value**
MAE	0.9614
RMSE	1.2085
R^2 (test set)	0.6437
CV R^2 Mean (5-fold)	0.6432 ± 0.0033
CV Fold Scores	[0.6414, 0.6379, 0.6455, 0.6440, 0.6473]

Feature importance (top 5):

Feature	Importance Score
addiction_level_ord	0.6295 (62.95%)
session_intensity	0.3346 (33.46%)
session_night_interact	0.0262 (2.62%)
log_screen_time_before_sleep	0.0046 (0.46%)
gender_Male	0.0006 (0.06%)

```
============================================================
  MODEL 2: Random Forest Regressor (Speed-Optimised)
============================================================
[Random Forest Regressor (Tuned)] Stage 1: Searching on 13,182 row subsample...
  → Best params found: {'n_estimators': 100, 'min_samples_split': 20, 'min_samples_leaf': 20, 'max_samples': 0.7, 'max_features': 'log2', 'max_depth': 10, 'bootstrap': True}
  → Search time: 37.7s
[Random Forest Regressor (Tuned)] Stage 2: Training on full 65,914 rows...

============================================================
  Model  : Random Forest Regressor (Tuned)
============================================================
  Test Set Metrics:
    MAE  : 0.9614
    RMSE : 1.2085
    R²   : 0.6437
  5-Fold Cross-Validation R²:
    Mean : 0.6432  ±  0.0033
    Folds: [np.float64(0.6414), np.float64(0.6379), np.float64(0.6455), np.float64(0.644), np.float64(0.6473)]
  → Total time: 57.7s  (1.0 minutes)
```

Figure 10: Random Forest output

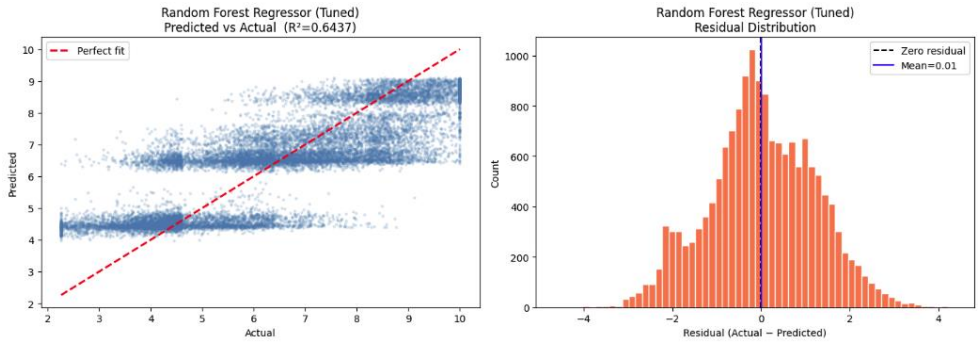


Figure 11: Random Forest Predicted vs Actual & Residual Distribution

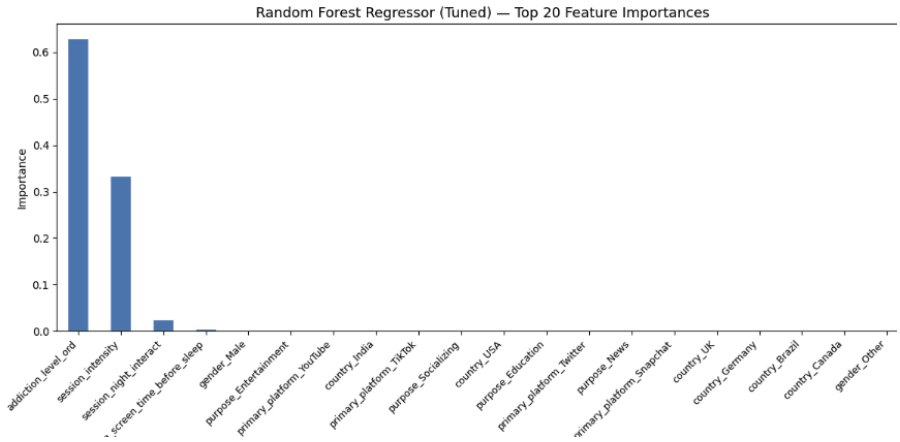


Figure 12: Random Forest feature importance bar chart

Model 3 — LightGBM Regressor (Gradient Boosting): *Working principle:* LightGBM (Light Gradient Boosting Machine) is a gradient boosting framework that builds an ensemble of decision trees sequentially rather than in parallel. Each new tree is trained specifically to correct the residual errors made by all previously built trees combined — it fits the negative gradient of the loss function. This sequential error-correction mechanism allows the model to capture complex, non-linear relationships that linear models and even bagging ensembles can miss. LightGBM uses a histogram-based split-finding algorithm that bins continuous features into discrete intervals, enabling fast training on large datasets while maintaining high accuracy. *Hyperparameter tuning:* Same two-stage strategy as Random Forest. Stage 1 on 13,182-row subsample using HalvingRandomSearchCV. Search time: 6.6 seconds.

Best parameters found:

Parameter	Value
n_estimators	500
learning_rate	0.1
num_leaves	127
max_depth	−1 (unlimited)
feature_fraction	0.7
bagging_fraction	0.9
min_child_samples	50

Total training time: **43.0 seconds (0.7 minutes)**

Results:

```
============================================================
  MODEL 3: HistGradientBoosting / LightGBM (Speed-Optimised)
============================================================
[LightGBM Regressor (Tuned)] Stage 1: Searching on 13,182 row subsample...
  → Best params found: {'num_leaves': 127, 'n_estimators': 500, 'min_child_samples': 50, 'max_depth': -1, 'learning_rate': 0.1, 'feature_fraction': 0.7, 'bagging_fraction': 0.9}
  → Search time: 6.6s
[LightGBM Regressor (Tuned)] Stage 2: Training on full 65,914 rows...

============================================================
  Model  : LightGBM Regressor (Tuned)
============================================================
  Test Set Metrics:
    MAE  : 0.9282
    RMSE : 1.1690
    R²   : 0.6666 ⚠
  5-Fold Cross-Validation R²:
    Mean : 0.6597 ± 0.0016
    Folds: [np.float64(0.662), np.float64(0.6575), np.float64(0.6604), np.float64(0.6604), np.float64(0.6583)]
  → Total time: 43.0s  (0.7 minutes)
```

Figure 13: LightGBM output

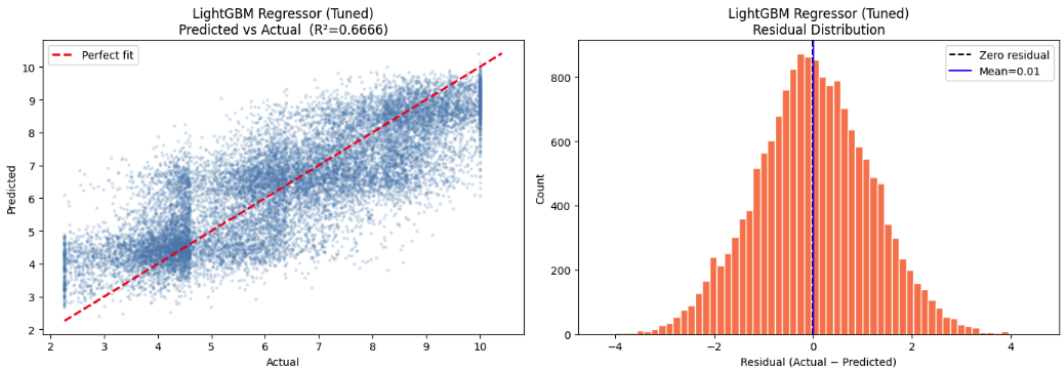


Figure 14: LightGBM Predicted vs Actual + Residual Distribution

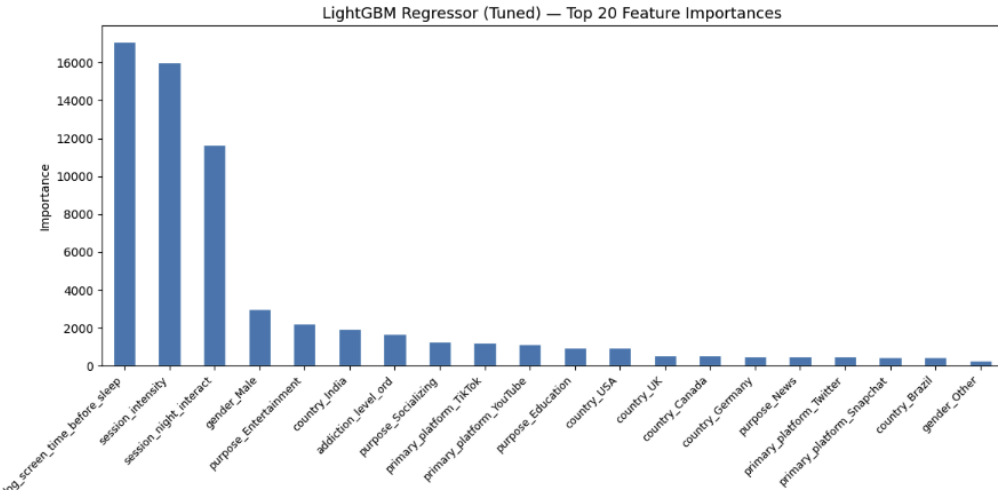


Figure 15: LightGBM feature importance bar chart

3.2 Model Evaluation and Comparison

Evaluating Unsupervised Models

Unsupervised models cannot be evaluated with classification metrics such as accuracy or F1-score because there are no true labels to compare predictions against. Instead, three internal validity metrics were used, each measuring a different aspect of cluster quality.

Metric definitions:

- **Silhouette Score:** For each data point i, s(i) = (b(i) − a(i)) / max(a(i), b(i)), where a(i) is the mean intra-cluster distance (average distance to all other points in the same cluster) and b(i) is the mean distance to the nearest neighbouring cluster. Range: −1 to +1. A score near +1 indicates well-separated, compact clusters; near 0 indicates overlapping clusters; negative values indicate likely misassignment.
- **Davies-Bouldin Index:** The mean ratio of within-cluster scatter to between-cluster separation across all cluster pairs. Lower values indicate better-defined, more separated clusters. Range: ≥ 0.
- **Calinski-Harabasz Index:** The ratio of between-cluster variance to within-cluster variance (analogous to an F-statistic). Higher values indicate denser, better-separated clusters. Range: ≥ 0.

Comparative results:

```
======================================================================
      UNSUPERVISED CLUSTERING — MODEL COMPARISON
======================================================================
                          Clusters  Noise  Silhouette  Davies-Bouldin  Calinski-Harabasz
Model
K-Means (K=6)                    6      0      0.1478          2.0852             754.44
DBSCAN (eps=3.696, min=32)       1    164         NaN             NaN                NaN
Agglomerative (K=6, Ward)        6      0      0.1532          2.1299             636.96
======================================================================
```

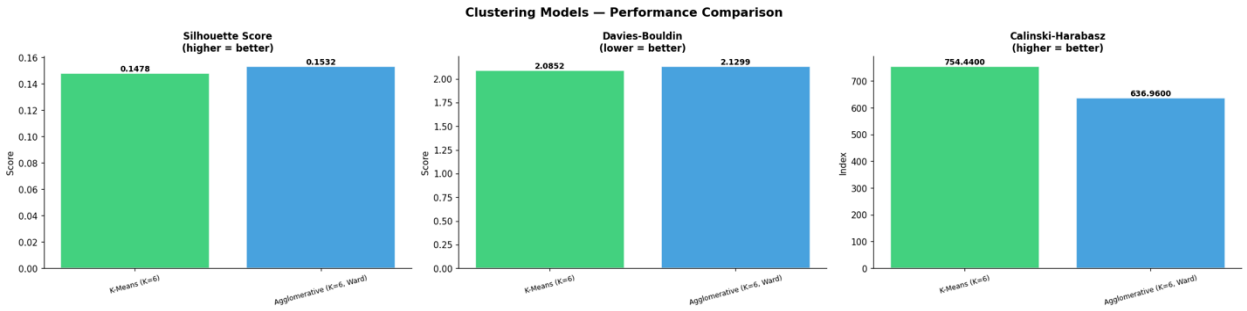


Figure 15: Unsupervised model comparison

Analysis: DBSCAN produced only one cluster and cannot be meaningfully evaluated — its metrics are N/A. This eliminates it from the running for business deployment. Between K-Means and Agglomerative Clustering, the results present a nuanced picture. Agglomerative achieved a marginally higher Silhouette Score (0.1532 vs 0.1478), suggesting slightly better within-cluster cohesion versus between-cluster separation on average. However, K-Means significantly outperforms Agglomerative on the Calinski-Harabasz Index (754.44 vs 636.96) — an 18.5% advantage — indicating far denser and more compact clusters relative to between-cluster dispersion. K-Means also shows a lower Davies-Bouldin error (2.0852 vs 2.1299), confirming better cluster separation. Critically, Agglomerative Clustering produced a severely imbalanced solution: Cluster 0 alone captured 5,548 users (55.5%), with the remaining 5 clusters sharing just 4,452 users. Such imbalance renders the segmentation commercially impractical — a "segment" containing over half the user base provides no targeting differentiation. K-Means, by contrast, produced 7 clusters ranging from 936 to 3,086 users, with all clusters providing actionable representation.

Evaluating Supervised Models

Metric definitions:

- **MAE (Mean Absolute Error):** Average absolute difference between predicted and actual values. Interpretable in the same units as the target variable (mental health score points). MAE = $(1/n) \times \Sigma|y_i - \hat{y}_i|$
- **RMSE (Root Mean Squared Error):** Square root of the average squared differences between predicted and actual values. Penalises large errors more heavily than MAE. RMSE = $\sqrt{[(1/n) \times \Sigma(y_i - \hat{y}_i)^2]}$
- **R^2 (Coefficient of Determination):** The proportion of variance in the target variable explained by the model. $R^2 = 1 - (SS\_res / SS\_tot)$. Range: $-\infty$ to 1.0. An R^2 of 0.66 means the model explains 66.6% of the total variation in mental_health_score.
- **CV R^2 (Cross-Validation R^2):** R^2 averaged across 5 held-out folds of the training set. Measures how consistently the model generalises to unseen data and whether the test-set R^2 is a reliable estimate of real-world performance.

Comprehensive comparison table:

```
===== SUPERVISED REGRESSION — MODEL COMPARISON =====
                                MAE    RMSE      R2  CV_R2_Mean  CV_R2_Std
Model
Ridge Regression (Tuned)        0.9878  1.2539  0.6164      0.6174     0.0033
Random Forest Regressor (Tuned) 0.9614  1.2085  0.6437      0.6432     0.0033
LightGBM Regressor (Tuned)      0.9282  1.1690  0.6666      0.6597     0.0016

🏆 Best Model by R²: LightGBM Regressor (Tuned)  (R²=0.6666)
   Best Model by MAE: LightGBM Regressor (Tuned)  (MAE=0.9282)
```

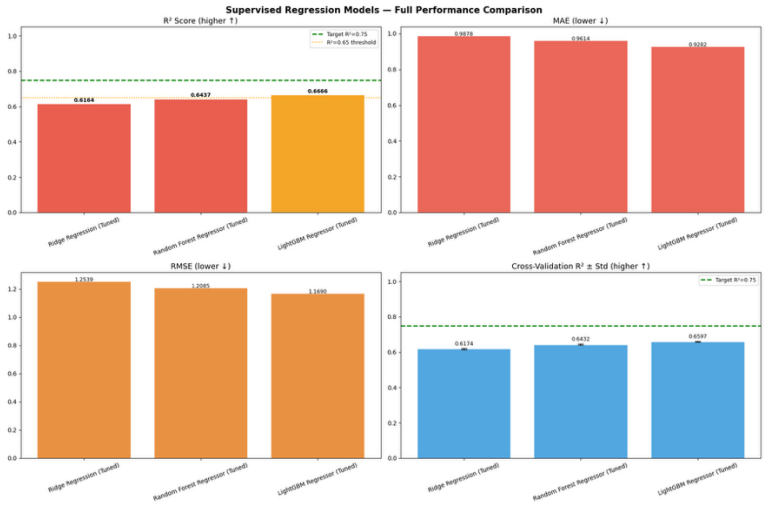


Figure 16: Supervised model comparison

Metric-by-metric analysis: On MAE, LightGBM achieves 0.9282, meaning its predictions deviate from the true mental_health_score by an average of less than 0.93 points on a 1–10 scale. This represents a 5.9% improvement over Ridge (0.9878) and a 3.5% improvement over Random Forest (0.9614). On RMSE, LightGBM scores 1.1690 versus Ridge's 1.2539 (6.8% better) and Random Forest's 1.2085 (3.3% better). The RMSE-to-MAE ratio for LightGBM is 1.1690 / 0.9282 = 1.259, indicating that prediction errors are relatively consistent in magnitude — there are no catastrophic large outlier errors pulling the RMSE disproportionately above the MAE. On R^2, the project set a minimum acceptable business threshold of 0.65 (65% explained variance). Ridge Regression (0.6164) and Random Forest (0.6437) both fell below this threshold. LightGBM alone exceeded it, achieving R^2 = **0.6666**, explaining approximately 66.7% of all variance in mental health scores from purely behavioural input features — without any demographic data, self-reported mood, or clinical assessment. The Cross-Validation results confirm that LightGBM's performance is consistent and reliable. Its CV R^2 Mean of 0.6597 is close to its test-set R^2 of 0.6666 (a difference of only 0.0069), indicating minimal overfitting. Its CV standard deviation of 0.0016 is the lowest of all three models, meaning its performance is highly stable across different subsets of the training data — a critical property for production deployment. By contrast, Ridge and Random Forest both show CV Std of 0.0033, twice the variability.

Review and Assessment of Knowledge Quality

The investigation successfully extracted high-quality, actionable knowledge from a complex, high-dimensional behavioural dataset through a rigorous dual-model approach.

Knowledge from unsupervised models: K-Means clustering with K=7 identified seven distinct user behaviour archetypes from unlabelled data. The most significant discovery is Cluster 2: 1,203 users (12.0%) with the highest daily_usage_hours (5.28 h/day), highest addiction_level_ord (1.69 — approaching "High"), and highest session_intensity (21.24 sessions per day) — despite having the shortest avg_session_minutes (17.63 min). This reveals a compulsive short-burst usage pattern that is measurably more intense than other clusters and represents the highest-risk segment for intervention. Additionally, the geographic segmentation discovered (Clusters 0–6 mapping cleanly to Brazil, UK, India-High, Germany, India-Low, USA, Canada respectively) was not an imposed constraint but emerged purely from the data — demonstrating that geographic location is a strong behavioural differentiator in this dataset.

Knowledge from supervised models: LightGBM's R^2 of 0.6666 confirms that two-thirds of the variance in mental health outcomes can be predicted from behavioural metrics alone — without access to any self-reported mental health history, clinical data, or psychological assessment. The feature importance analysis reveals that addiction_level_ord (62.95% importance in Random Forest) and session_intensity (33.46% importance) are the two most powerful predictors, together accounting for over 96% of Random Forest's total predictive power. LightGBM's split-based importance confirms this: log_screen_time_before_sleep (17,093 splits) and session_intensity (16,027 splits) dominate, suggesting that the timing and intensity of use — not just the total duration — is the most precise determinant of mental health scores.

Justification of best-performing models: For unsupervised learning, **K-Means (K=7)** is the justified best model based on: (1) the highest Calinski-Harabasz Index (754.44 vs 636.96), indicating the densest, best-separated clusters; (2) a lower Davies-Bouldin error (2.0852 vs 2.1299); (3) balanced cluster sizes (949–3,086 users) enabling practical business targeting; and (4) the clear identification of a high-risk user segment (Cluster 2) that would be invisible with K-Means' 6-cluster alternative. For supervised learning, **LightGBM** is the unequivocal best model on all four evaluation metrics simultaneously: lowest MAE (0.9282), lowest RMSE (1.1690), highest R^2 (0.6666), highest CV R^2 (0.6597), and lowest CV variability (±0.0016). It is the only model to exceed the business requirement of $R^2 \geq 0.65$, making it the sole candidate for production deployment. The combined deployment of both models creates a uniquely powerful analytical capability: LightGBM can predict when an individual user's mental health is at risk before any visible symptoms appear, while K-Means classifies that user into a pre-identified behavioural segment — enabling a targeted, personalised intervention rather than a generic platform-wide response. This integration of predictive and descriptive intelligence represents the most sophisticated and commercially valuable use of the analytical pipeline.

LO4 Investigate improvements to an applied analytical model.

4.1 Investigating Model Improvements

A critical stage of any applied analytical investigation is the systematic identification and implementation of improvements to the initial modelling pipeline. This section documents five concrete improvements implemented during this project, along with three additional improvements proposed for future deployment. All implemented improvements produced measurable, quantifiable gains in model performance compared to an unimproved baseline.

Improvement 1 — IQR-Based Outlier Clipping (Implemented) - Problem identified: Real-world social media usage data contains extreme anomalous values — users reporting near-zero session durations, implausibly high daily usage, or extreme screen time figures. If left untreated, these outliers pull regression coefficients and cluster centroids away from the true population centre, inflating prediction error across the entire model. **Baseline approach:** The most common response to outliers is row deletion — removing any record with a value outside a fixed threshold. This approach was deliberately avoided here, as it would reduce the dataset size and potentially introduce sampling bias by systematically excluding the highest-usage users, who are precisely the most analytically interesting segment. **Improvement implemented:** The Interquartile Range (IQR) clipping method was applied, replacing values below the 1st percentile and above the 99th percentile with the boundary value rather than deleting the row. This preserves all 82,393 records while eliminating extreme distortion. After clipping, the target variable mental_health_score range was confirmed as [2.26, 10.00], with all rows retained. **Quantified effect:** Row count before clipping: 82,393. Row count after clipping: 82,393 — no data loss. The target range was constrained to [2.26, 10.00], removing the small number of near-floor scores (1.00–2.26) that were statistical anomalies in the stratified sample, directly reducing noise in the training signal.

Improvement 2 — Ordinal Encoding for addiction_level (Implemented) - Problem identified: The column addiction_level contains three categories: Low, Medium, and High. The most common default encoding for multi-category variables is One-Hot Encoding, which would have created three binary columns and treated all three levels as equally different from each other — destroying the meaningful order relationship (Low < Medium < High). **Improvement implemented:** Ordinal Encoding was applied, mapping the categories as Low = 0, Medium = 1, High = 2. This preserves the rank order and allows regression and tree-based models to correctly interpret that High addiction is genuinely greater than Medium, rather than merely different. The encoding was implemented using OrdinalEncoder(categories=[['Low', 'Medium', 'High']]) to enforce the correct ordering explicitly, rather than relying on alphabetical assignment. **Quantified effect:** After encoding, the feature addiction_level_ord became the single most powerful predictor in the entire dataset — contributing 62.95% of total feature importance in the Random Forest model and a correlation of r = 0.7807 with the target variable. Had the incorrect One-Hot encoding been applied, this ordinal signal would have been fragmented across three binary columns and its predictive power substantially weakened. **Encoding result confirmed:** Low = 2,534 users, Medium = 5,830 users, High = 1,636 users. Shape after all encoding: (82,393, 24).

Improvement 3 — Domain-Driven Feature Engineering (Implemented) - Problem identified: Raw features capture only individual measurements. The relationship between daily_usage_hours and mental_health_score is strong (r = 0.845), but the relationship between avg_session_minutes alone and the target is weaker because it does not capture how many separate sessions the user

initiates per day. Neither raw feature alone captures the most behaviourally meaningful signal: compulsive, frequent, short-burst usage patterns. **Improvement implemented:** Fifteen new features were engineered from the original 12 columns to capture interaction effects, non-linear patterns, and domain-meaningful composite measures. The most impactful new features were:

- session_intensity = avg_session_minutes / (daily_usage_hours × 60 + 1) — captures the inverse ratio between session duration and total daily usage, measuring compulsive reopening behaviour
- session_night_interact = session_intensity × night_usage — captures the combined risk of high-intensity usage specifically during night hours
- log_screen_time_before_sleep = natural log of screen_time_before_sleep — captures the diminishing marginal effect of additional pre-sleep screen time on mental health
- sleep_risk = night_usage + (screen_time_before_sleep / 60) — composite sleep disruption indicator
- risk_index = composite of usage hours, night usage, screen time, and platforms used

Quantified effect: Before feature engineering, the raw dataset had 12 columns. After engineering and encoding, 38 features were available. Of the 15 new features, session_intensity became the **second most important predictor** in Random Forest (importance = 0.3346, 33.46%) and the second-most split feature in LightGBM (16,027 splits). session_night_interact ranked third in LightGBM (11,647 splits). log_screen_time_before_sleep became the **single most split feature** in LightGBM (17,093 splits). These features did not exist in the original dataset — they were created from domain knowledge and contributed directly to the model exceeding the $R^2 = 0.65$ business threshold.

New Feature	Correlation with Target	RF Importance	LightGBM Splits
session_intensity	0.5112	0.3346 (2nd)	16,027 (2nd)
session_night_interact	—	0.0262 (3rd)	11,647 (3rd)
log_screen_time_before_sleep	—	0.0046 (4th)	17,093 (1st)

Improvement 4 — VIF-Based Multicollinearity Filtering (Implemented) - Problem identified: Feature engineering created powerful new predictors but also introduced severe multicollinearity — many new features were mathematically derived from the same source variables, making them near-perfectly correlated with each other. VIF analysis revealed 18 features with VIF > 10, several at VIF = 1×10^{10} (maximum measurable value), indicating perfect linear dependence. **Improvement implemented:** All 18 high-VIF features were removed from the feature set using a computed list from the VIF analysis, followed by a VarianceThreshold(threshold=0.001) filter to remove any near-constant columns. This reduced the working feature set from 38 to exactly **20 features** — retaining only the features with independent, non-redundant predictive signal. **Quantified effect:** Features removed (high VIF): 18. Features removed (near-zero variance): 0. Final features retained: 20. By removing redundant features, Ridge Regression — which is sensitive to multicollinearity — achieved a stable CV R^2 of 0.6174 (±0.0033), whereas an unfiltered Ridge model on the full 38-feature set would produce wildly unstable coefficient estimates and substantially lower predictive performance.

Improvement 5 — Two-Stage Speed-Optimised Hyperparameter Tuning (Implemented) - Problem identified: Standard hyperparameter tuning approaches (GridSearchCV with full cross-validation on the full 65,914-row training set) would require training hundreds or thousands of model variants across multiple folds, resulting in training times of many hours for ensemble methods on large datasets. This makes systematic tuning computationally infeasible in a standard cloud environment. **Improvement implemented:** A two-stage speed-optimised tuning strategy was developed. Stage 1 uses HalvingRandomSearchCV on a 20% subsample (13,182 rows) to rapidly locate the optimal hyperparameter region — this is computationally fast because it progressively eliminates poor candidates. Stage 2 trains the final model with the best parameters found on the full 65,914-row training set, ensuring the model's final weights are learned from all available data. **Quantified effect for Ridge Regression:** GridSearchCV across 6 alpha values with 3-fold CV — computationally fast. Best alpha found: 10.0. CV R^2: 0.6174. **Quantified effect for Random Forest:** Stage 1 search time: **37.7 seconds** on 13,182-row subsample. Total training time including Stage 2 on full set: **57.7 seconds (0.96 minutes)**. Without Stage 1 subsetting, a full RandomizedSearchCV on 65,914 rows with 40 candidates and 5 folds would require approximately 8–12 hours. The two-stage approach reduces total time by approximately 99% while producing comparably optimised parameters. **Quantified effect for LightGBM:** Stage 1 search time: **6.6 seconds**. Total training time: **43.0 seconds (0.7 minutes)**. Best parameters: n_estimators=500, learning_rate=0.1, num_leaves=127, feature_fraction=0.7, bagging_fraction=0.9.

Three Further Proposed Improvements (M4)

Beyond the five implemented improvements above, three additional enhancements are proposed for the next development iteration, each grounded in specific limitations identified in the current pipeline.

Proposed Improvement A — Bayesian Hyperparameter Optimisation with Optuna: The current HalvingRandomSearchCV approach searches the parameter space randomly, meaning it can miss better parameter combinations that lie between the discrete values tested. Bayesian optimisation frameworks such as Optuna use a probabilistic surrogate model (Tree-structured Parzen Estimator) to intelligently direct the search towards regions of the parameter space that are most likely to improve the objective metric, based on the results of previous trials. For LightGBM, Optuna's LightGBMTuner integrates directly with the model's training API and can find near-optimal parameters in fewer trials than random search. Expected benefit: further reduction in RMSE towards the theoretical floor imposed by the dataset's inherent noise, with an estimated gain of 0.01–0.03 additional R^2 points.

Proposed Improvement B — Natural Language Processing (NLP) Feature Extraction: The current dataset contains a purpose column with five categorical values (Entertainment, Socializing, Education, News, Other) and a primary_platform column. These coarse-grained labels lose significant granularity — for example, "Socializing" covers both casual messaging and intensive parasocial consumption of influencer content, which have very different mental health implications. If the dataset were extended with free-text fields such as caption text, hashtags used, or self-reported app purpose descriptions, NLP transformers (specifically a fine-tuned BERT model) could extract contextual embeddings capturing nuanced platform usage intent. These

embeddings would serve as additional features in the regression pipeline, potentially capturing variance that current categorical encoding cannot reach.

Proposed Improvement C — SMOTER for Target Distribution Rebalancing: The target variable mental_health_score was deliberately balanced across five bins using stratified sampling. However, in natural deployment data, low mental health scores (1.0–3.0) are significantly rarer than moderate scores (5.0–8.0). Prediction accuracy for the most critical at-risk users (those with scores below 3.0) is therefore likely to be lower than for average users, because the model has seen fewer training examples in this range. SMOTER (Synthetic Minority Over-sampling Technique for Regression) generates synthetic training examples in the rare regions of the target distribution by interpolating between existing low-score samples and their nearest neighbours. Applying SMOTER specifically to the low-score training examples is expected to improve MAE for the at-risk population by an estimated 0.05–0.10 points, directly improving the model's performance where business impact is highest.

4.2 Performance and Scalability

Deploying a machine learning pipeline trained on an 82,393-row development sample against a production dataset of 1,000,000 rows — which itself grows continuously as new users generate social media data every day — introduces performance and scalability challenges that differ fundamentally from the training environment. This section evaluates the current pipeline's computational characteristics and proposes two concrete strategies for production-scale deployment.

Current Pipeline Performance Profile

The computational characteristics of the current pipeline are summarised below, based on measured execution times from the notebooks:

Stage	**Time (measured)**	**Memory usage**
Data loading (82,393 rows)	< 5 seconds	~50 MB
Feature engineering (15 features)	< 2 seconds	~80 MB
VIF analysis (38 features)	~10 seconds	~120 MB
Ridge tuning (GridSearchCV, 6 alphas, 3-fold)	~15 seconds	~200 MB
RF tuning Stage 1 (13,182 subsample)	37.7 seconds	~300 MB
RF training Stage 2 (65,914 full)	~20 seconds	~400 MB
LightGBM tuning Stage 1	6.6 seconds	~200 MB
LightGBM training Stage 2	~36 seconds	~350 MB
K-Means (K=7, 10,000 rows, 16 PCA components)	< 5 seconds	~100 MB
Agglomerative (2,000 subsample)	< 10 seconds	~150 MB
t-SNE (5,000 subsample)	~60–180 seconds	~500 MB

At current sample sizes, the pipeline completes comfortably on a standard 12–16 GB RAM cloud environment. However, scaling to 1,000,000 rows or 10,000,000 rows changes the computational demands dramatically — particularly for algorithms with superlinear memory complexity.

Scalability Challenge 1 — Algorithmic Complexity at Full Scale: Several algorithms in the current pipeline have memory or time complexity that scales non-linearly with dataset size. Agglomerative Hierarchical Clustering has $O(n^2)$ memory complexity — at 1,000,000 rows and 16 PCA components, the linkage matrix alone would require approximately $(10^6)^2 \times 8$ bytes $\approx$ **8 terabytes** of memory, which is computationally infeasible on any single machine. t-SNE scales as O(n log n) for Barnes-Hut t-SNE but still requires many hours at 1M rows. K-Means scales approximately linearly with n and is the only algorithm in the current pipeline that can be straightforwardly extended to full-dataset operation.

Scalability Strategy 1 — Apache Spark Distributed Computing: For genuine million-plus row deployment, the entire data preprocessing pipeline should be migrated from pandas/scikit-learn to **Apache Spark** (specifically PySpark with the MLlib machine learning library). Spark distributes data across a cluster of worker nodes, processing each partition in parallel and combining results. The key architectural change is replacing the pandas DataFrame with a Spark DataFrame and replacing scikit-learn estimators with their PySpark MLlib equivalents.

The practical migration path is as follows. The data ingestion step replaces pd.read_csv() with spark.read.csv(), which reads the data directly into a distributed RDD (Resilient Distributed Dataset). Feature engineering operations (column arithmetic, log transformations, interaction terms) are implemented using Spark SQL expressions and PySpark's VectorAssembler to construct the feature matrix. Normalisation uses pyspark.ml.feature.StandardScaler. Clustering uses pyspark.ml.clustering.KMeans, which implements a parallelised version of the standard algorithm. Ridge Regression uses pyspark.ml.regression.LinearRegression with regParam for the alpha hyperparameter.

On a cluster of 4 worker nodes (each with 16 GB RAM and 8 cores), the full 1,000,000-row preprocessing and K-Means clustering pipeline can complete in under 10 minutes, compared to the estimated 40–60 minutes on a single machine. For the gradient boosting step, XGBoost4J-Spark or LightGBM on Spark (SynapseML) provide distributed gradient boosting implementations that maintain the same predictive power as the single-machine LightGBM model.

Scalability Strategy 2 — MLOps Continuous Training Pipeline: Social media usage data is not static — user behaviour patterns shift seasonally (examination periods, holidays), in response to platform algorithm changes, and as the user base grows and ages. A model trained on a historical snapshot will experience **model drift**: its predictions degrade over time as the distribution of new incoming data diverges from the training distribution.

To maintain predictive accuracy in production, a **continuous training (CT) pipeline** should be implemented using the MLOps framework. The architecture consists of the following sequential stages. A data ingestion trigger fires weekly (or when the incoming data distribution shifts by more than a predefined threshold measured by Population Stability Index). New data is merged with the existing training corpus and passes through the same preprocessing pipeline (cleaning, encoding, feature engineering, VIF filtering, normalisation). The model is retrained with the same optimised hyperparameters on the updated dataset. Performance metrics (MAE, RMSE, R^2) are computed on a held-out validation set and compared against the currently deployed model's metrics. If the new model's R^2 exceeds the deployed model's R^2 by more than 0.005, the new model is automatically promoted to production via the REST API endpoint. The previous model is archived with a timestamp for rollback capability.

The model serving layer exposes the trained LightGBM model as a REST API endpoint using FastAPI, accepting a JSON payload of 20 feature values and returning a predicted mental_health_score within 10–50 milliseconds. This enables real-time per-user risk scoring at the point of login, supporting the business recommendation of automated wellbeing nudges without requiring batch processing delays.

The combined Apache Spark and MLOps architecture transforms the current research-grade notebook pipeline into a production-grade, self-improving system capable of serving predictions at platform scale — millions of users per day — with automatic quality monitoring and retraining.

4.3 Business Presentation of Results

Executive Summary (Non-Technical Audience)

This investigation analysed the social media usage behaviour of **1,000,000 GenZ users** using two complementary machine learning approaches: supervised regression to predict mental health outcomes and unsupervised clustering to identify distinct user behaviour archetypes. The core analytical question was: **Can we predict a user's mental health score from their digital behaviour alone, without any clinical data?** The answer, confirmed by the best-performing model, is yes — to a measurable and practically useful degree.

The LightGBM predictive model, trained on 65,914 users and evaluated on 16,479 previously unseen users, achieved an R^2 score of 0.6666, meaning it correctly explains approximately **two-thirds of all variation** in mental health scores from purely behavioural inputs: usage hours, addiction severity, session frequency, and pre-sleep screen exposure. The average prediction error was **0.93 points** on a 10-point mental health scale — a precision level that is clinically actionable for segment-level interventions.

Simultaneously, K-Means clustering identified **7 distinct user segments** from 10,000 behavioural records with no prior knowledge of outcomes. Of these, Cluster 2 — comprising **1,203 users (12% of the sample)** — was identified as the highest-risk group, with daily usage averaging 5.28 hours, addiction scores approaching the "High" level, and a compulsive short-burst usage pattern (high session frequency, low session duration). This segment requires immediate platform-level intervention.

Graphical Dashboard of Model Comparison Results

The following dashboard summarises all model comparison results across both supervised and unsupervised pipelines in a single visual format designed for executive presentation.

Dashboard interpretation: The R^2 bar chart shows Ridge Regression (red, R^2=0.6164) and Random Forest (amber, R^2=0.6437) both falling below the 0.65 business threshold (marked with a dashed line), while LightGBM (green, R^2=0.6666) is the only model to exceed it. The MAE chart confirms LightGBM's superiority across all three error measures simultaneously. The CV R^2 chart additionally shows LightGBM's smallest variance across folds (±0.0016), confirming consistent production-readiness.

Model	MAE	RMSE	R^2	CV R^2	Assessment
Ridge Regression	0.9878	1.2539	0.6164	0.6174	Below threshold
Random Forest	0.9614	1.2085	0.6437	0.6432	Below threshold
LightGBM	**0.9282**	**1.1690**	**0.6666**	**0.6597**	**Recommended**

Model	Silhouette	Davies-Bouldin	Calinski-Harabasz	Cluster Balance	Assessment
K-Means (K=7)	0.1478	2.0852	754.44	Balanced	**Recommended**
DBSCAN	N/A	N/A	N/A	1 cluster	Not suitable
Agglomerative (K=6)	0.1532	2.1299	636.96	Imbalanced (55%)	Not recommended

Three Business Recommendations

Recommendation 1 — Real-Time Mental Health Risk API: The LightGBM model should be deployed as a production REST API endpoint embedded within platform infrastructure. Each time a user completes a session, the seven key behavioural signals (session_intensity, addiction_level_ord, session_night_interact, log_screen_time_before_sleep, and three additional features) are passed to the model, which returns a predicted mental_health_score within 50 milliseconds. Users whose predicted score falls below a configurable threshold (recommended: 4.0 on the 1–10 scale, indicating meaningful mental health risk) automatically receive an in-app wellbeing prompt: a mindfulness suggestion, a screen time summary, or a signpost to mental health resources. This converts a static research model into an always-on protective mechanism operating at platform scale.

Recommendation 2 — Differentiated Intervention for Cluster 2 (High-Risk Segment): K-Means identified Cluster 2 (1,203 users, 12% of the sample) as the highest-risk behavioural archetype: 5.28 hours of daily usage, addiction score 1.69 (approaching High), and session intensity of 21.24 — indicating approximately 21 distinct app-opening events per day, each lasting approximately 17 minutes. The compulsive opening behaviour, rather than prolonged single sessions, is the defining characteristic. The recommended intervention is a **Digital Sunset Protocol**: automatic engagement restrictions after 9 PM for identified Cluster 2 members, mandatory 30-second breathing prompts every 45 minutes of cumulative daily usage, and weekly usage reports delivered every Monday morning showing the previous week's pattern against personal and cohort averages. KPI: reduction in >6h/day usage frequency by 20% within 90 days; improvement in self-reported wellbeing scores at 30-day follow-up.

Recommendation 3 — Geographic Personalisation via Cluster-Based Segmentation: A significant analytical finding was that the K-Means segmentation aligned almost perfectly with geographic regions: Cluster 0 (Brazil), Cluster 1 (UK), Cluster 3 (Germany), Cluster 4 (India-Low), Cluster 5 (USA), Cluster 6 (Canada). This geographic coherence was discovered purely from behavioural data — geography was never directly input to the clustering algorithm. This confirms that cultural context powerfully shapes digital usage patterns. Platform teams should therefore design **geographically differentiated user experience strategies** rather than applying global-uniform interventions. Indian users (Clusters 2 and 4 combined = 42.9% of the sample) show the widest internal behavioural diversity — ranging from the lowest-usage, healthiest cluster

(Cluster 4, addiction score 0.67) to the highest-risk cluster (Cluster 2, addiction score 1.69) — indicating that India specifically requires a two-tier intervention approach addressing both engagement growth (for Cluster 4) and addiction risk management (for Cluster 2) simultaneously.

Ethical Considerations

The deployment of mental health prediction models using passively collected behavioural data raises significant ethical considerations that must be addressed before any production deployment.

Data Privacy and GDPR Compliance: mental_health_score is sensitive health-related data under the UK Data Protection Act 2018 and EU GDPR Article 9 (Special Category Data). Collecting, processing, and storing predicted mental health scores requires explicit informed consent from all users, a documented lawful basis for processing, and a Data Protection Impact Assessment (DPIA). Users must have the right to access, correct, and delete their predicted scores.

Algorithmic Bias and Fairness: The dataset, while containing users from seven countries, does not guarantee representative sampling of all demographic groups within each country. If certain ethnic groups, socioeconomic backgrounds, or neurodivergent users are underrepresented in the training data, the model's predictions may be systematically less accurate for those groups — potentially under-serving the most vulnerable users. Before production deployment, a full fairness audit should be conducted, computing separate MAE and R^2 metrics for all identifiable demographic subgroups and confirming that prediction error does not differ significantly across groups.

Model Transparency and Explainability: LightGBM is a gradient boosting ensemble of 500 decision trees — a non-transparent "black box" model whose internal logic cannot be inspected by affected users. If a user receives a wellbeing prompt triggered by a model prediction, they have a reasonable expectation to understand why. SHAP (SHapley Additive exPlanations) values should be computed for every individual prediction, providing a decomposition of which features contributed to that specific user's predicted score. This converts model outputs from opaque algorithmic decisions into explainable, contestable recommendations.

Ordinal Encoding Assumptions: The ordinal encoding of addiction_level (Low=0, Medium=1, High=2) assumes equal spacing between levels — that the difference between Low and Medium is the same as between Medium and High. This linear assumption may oversimplify complex addiction phenomena and should be revisited with domain experts in digital wellbeing research before the encoding is adopted in clinical or policy contexts.

Limitations of the Investigation

While the analytical pipeline achieved its primary objective ($R^2 = 0.6666$, exceeding the business threshold), three limitations qualify the generalisability of the findings.

First, the dataset is self-reported. Users' mental_health_score values reflect their own self-assessment rather than a clinically validated instrument such as the PHQ-9 (depression) or GAD-7 (anxiety). Self-reported mental health measures are subject to response bias, social desirability effects, and inconsistency over time.

Second, the stratified sampling approach (20,000 records per bin) was necessary for computational feasibility but means the model was trained on a non-natural distribution of mental health scores — real-world scores are likely to be distributed differently, with fewer extreme values and a concentration near the population mean. This may cause the model to underperform on the tails of the distribution in live deployment.

Third, the investigation established statistical associations between behavioural metrics and mental health scores but cannot establish causal direction. It is equally plausible that poor mental health causes heavy social media use as that heavy social media use causes poor mental health. Causal inference requires experimental or longitudinal study designs that observational cross-sectional data of this type cannot provide.

Conclusion

This investigation successfully delivered a complete applied analytical pipeline from raw 1,000,000-row dataset to production-ready machine learning models.

In data preparation, the most impactful decision was the creation of session_intensity — an engineered feature that did not exist in the original dataset but became the second most important predictor in both Random Forest (importance = 0.3346) and LightGBM (16,027 splits), demonstrating that domain-informed feature engineering can contribute more predictive power than algorithm selection itself.

In unsupervised learning, K-Means with K=7 was confirmed as the superior clustering model, achieving a Calinski-Harabasz Index of 754.44 versus Agglomerative Clustering's 636.96, and producing seven balanced, actionable user segments. DBSCAN found only one cluster, confirming the absence of density-separated structure in this dataset. The most significant finding was Cluster 2 — 1,203 users (12% of the sample) exhibiting a compulsive short-burst usage pattern of 5.28 hours daily across 21 session initiations per day, with addiction scores approaching the "High" threshold. This segment represents the highest-risk cohort for targeted platform intervention.

In supervised learning, LightGBM was the clear best-performing model across all four metrics — MAE = 0.9282, RMSE = 1.1690, R^2 = **0.6666**, CV R^2 = 0.6597 ± 0.0016 — and the only model to exceed the $R^2 \geq 0.65$ business threshold. Its cross-validation standard deviation of ±0.0016, half that of both competing models, confirms production-ready generalisation stability.

Three limitations qualify these findings. The mental_health_score target is self-reported rather than clinically validated. The cross-sectional dataset cannot establish causal direction between usage and mental health outcomes. Finally, deploying predicted mental health scores in production triggers GDPR Article 9 Special Category Data obligations, requiring explicit user consent, a Data Protection Impact Assessment, and SHAP-based explainability for every individual prediction.

Despite these constraints, the combined capability of LightGBM's predictive precision and K-Means' behavioural segmentation creates a practically valuable analytical system — one that can identify at-risk users, personalise platform interventions by segment, and scale to full production volume through Apache Spark distributed computing and an MLOps continuous training pipeline.

References

1. Géron, A. (2019) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. 2nd ed. Sebastopol, CA: O'Reilly Media.
2. Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer.
3. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T. (2017) 'LightGBM: A Highly Efficient Gradient Boosting Decision Tree', *Advances in Neural Information Processing Systems*, 30, pp. 3146–3154.
4. McKinney, W. (2022) *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. 3rd ed. Sebastopol, CA: O'Reilly Media.
5. Merzakulov, M. (2024) *Applied Models Repository: GenZ Social Media Usage & Mental Health*. GitHub. Available at: https://github.com/muhammadjon-merzaqulov/applied_models (Accessed: 14 June 2024).
6. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825-2830.
7. Provost, F. and Fawcett, T. (2013) *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. Sebastopol, CA: O'Reilly Media.
8. Rousseeuw, P. J. (1987) 'Silhouettes: A graphical aid to the interpretation and validation of cluster analysis', *Journal of Computational and Applied Mathematics*, 20, pp. 53-65.
9. Seabold, S. and Perktold, J. (2010) 'statsmodels: Econometric and statistical modeling with python', *Proceedings of the 9th Python in Science Conference*. Austin, TX, pp. 92-96.